Sécurisation des échanges d'informations entre deux ordinateurs

Maxime LE BLAN

28798

Système d'étude

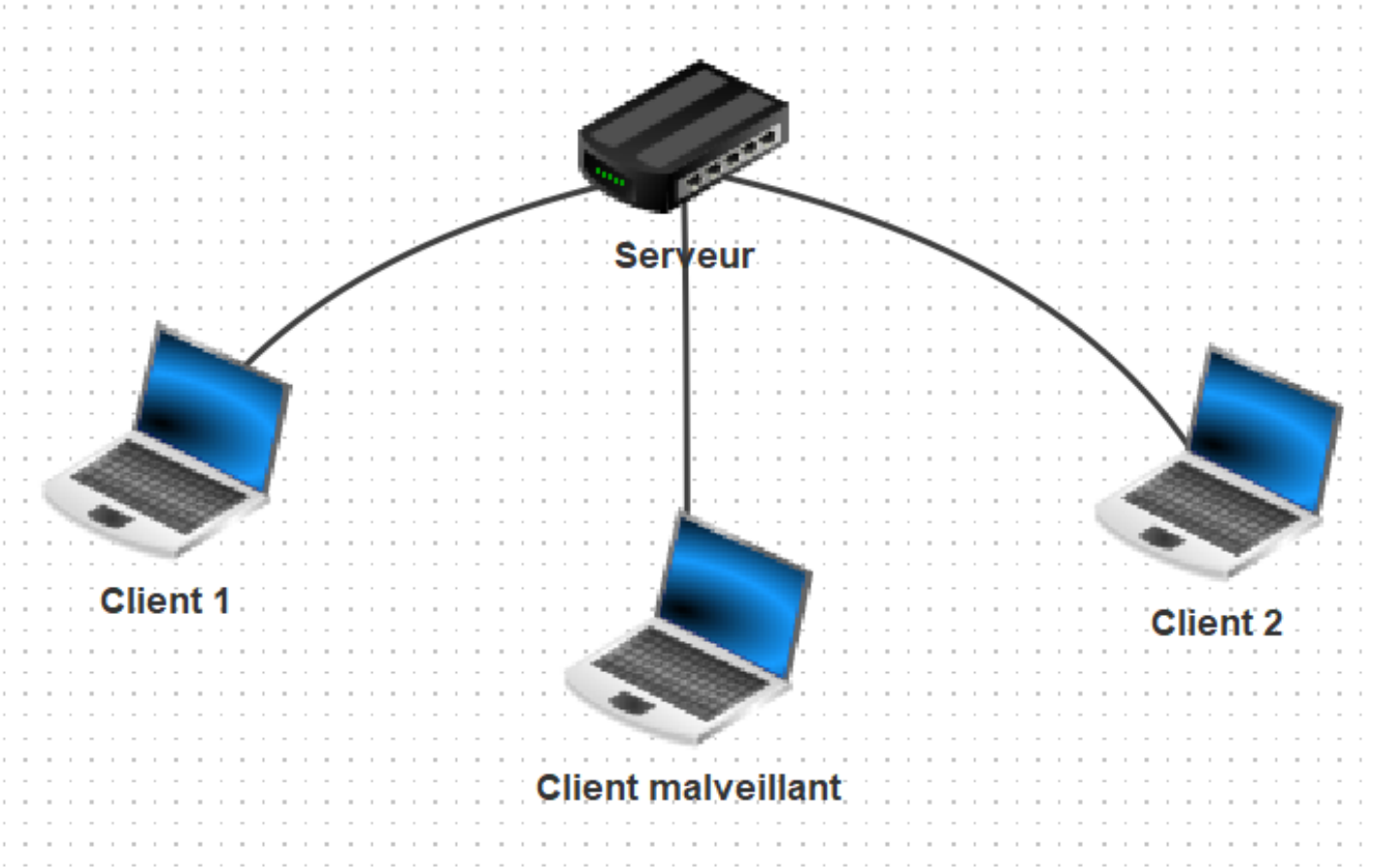


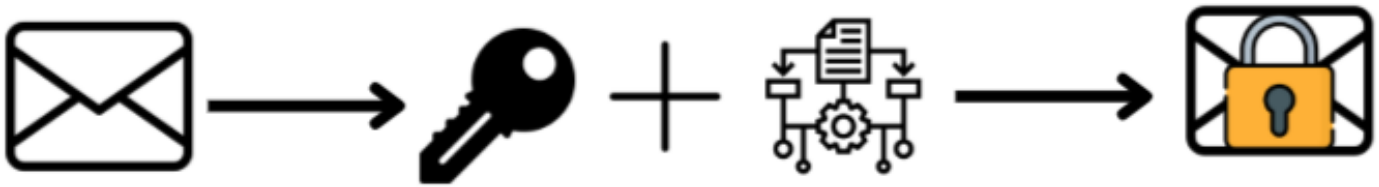
Figure: Representation imagée du système étudié

Sommaire

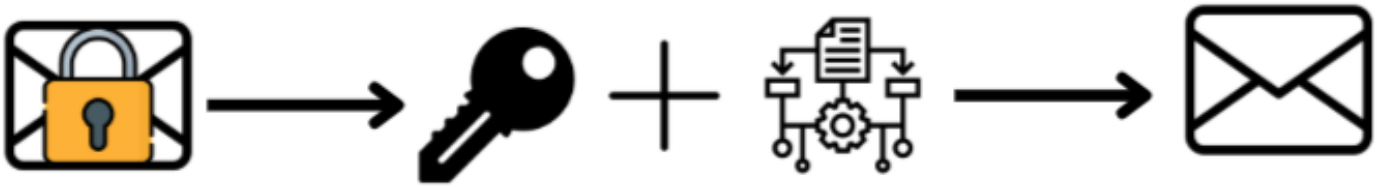
1. Définitions des notions et méthodes d'implémentations
2. Premier modèle de sécurisation avec l'algorithme RSA
3. Amélioration du premier modèle avec l'algorithme AES
4. Tentative de conservation de l'intégrité des communications

Vocabulaire

Chiffrement :



Déchiffrement :



Définitions

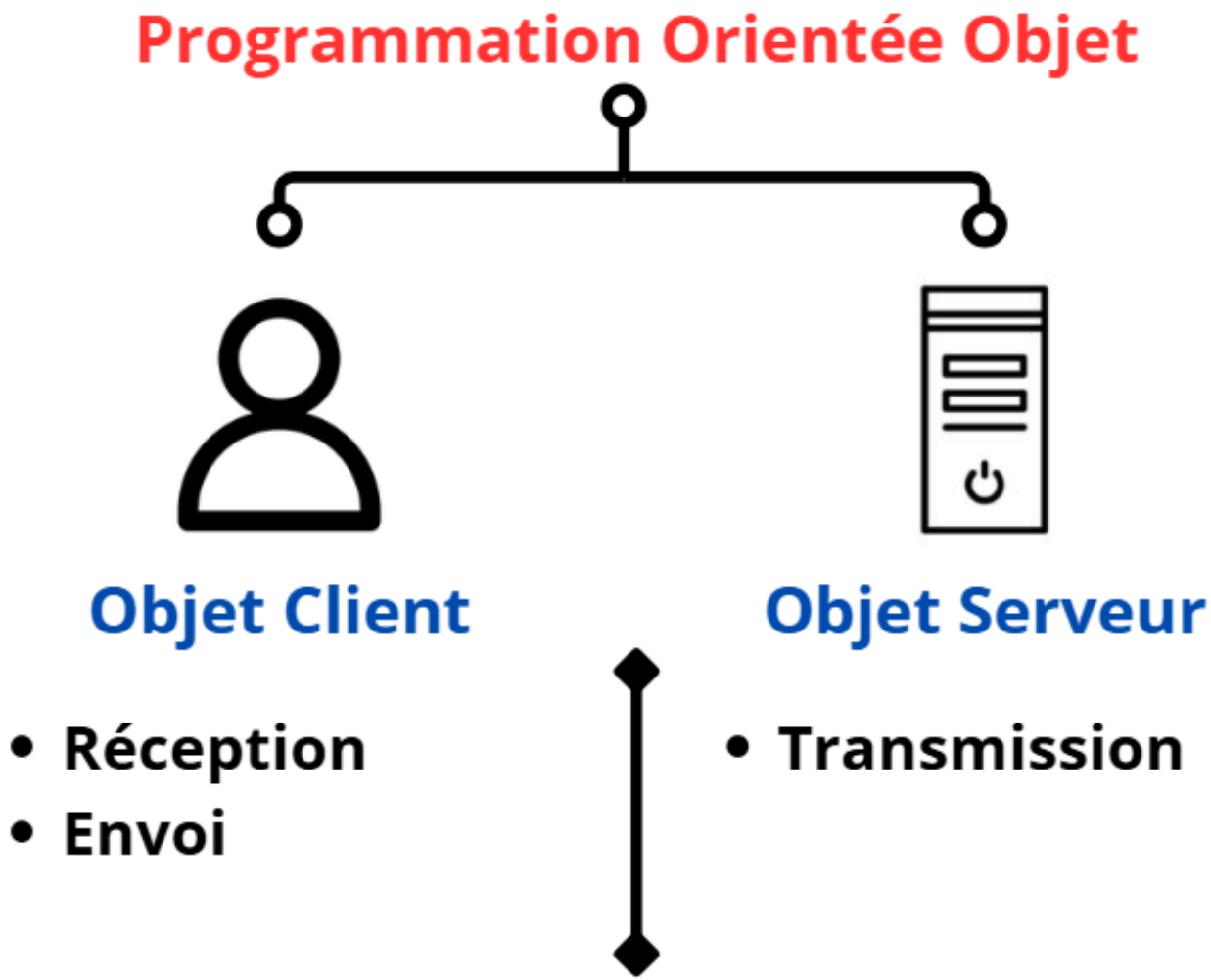
Chiffrement symétrique :



Chiffrement asymétrique :



Implémentation



Implémentation

1. Problèmes :
   - Programmation séquentielle ne permet pas d'envoyer et de recevoir des messages en même temps
   - Envoie de données multiples ne peut se faire qu'avec des chaines de caractères
2. Solutions :
   - Utilisation de fils d'exécutions
   - Fonctions de conversion de liste de données chiffrées en chaines de caractères

Algorithme RSA

1. Algorithme de chiffrement asymétrique
2. Se base sur deux conjectures :
   - "casser" RSA par force brute nécessite de trouver la décomposition en facteurs premiers d'un nombre entier de grande taille
   - les algorithmes existants ont une complexité exponentielle en la taille de la clé

Exemple

- Choix de 2 nombres premiers p et q

```
"C:\Users\maxim\OneDrive\Documents\Informatique\Python (prgms pers
Message à chiffrer :  32
Choix de p :  99999373
Choix de q :  99999437
```

- On pose $n = p \times q$ et $\phi_n = (p-1) \times (q-1)$

```
Valeur de n = p * q :  9999881000353001
Valeur de phi_n = (p-1) * (q-1) :  9999880800354192
```

- On prend e un nombre premier avec ϕ_n et on calcule d l'inverse modulaire de e modulo ϕ_n

```
Pour e tel que PGCD(e, phi_n) = 1, on a e :  5
Valeur de d inverse modulaire de e mod phi_n :  3999952320141677
```

- On chiffre m en calculant $m^e \pmod{n}$ avec (n, e) la clé publique et on déchiffre m' en calculant $m'^d \pmod{n}$ avec d la clé privée

```
Clé publique (n, e) :  (9999881000353001, 5)
Clé privée d :  3999952320141677
Chiffrement de m :  33554432
Déchiffrement de m' :  32
```

Figure: Exécution de l'algorithme RSA sur un exemple

Protocole du modèle n°1

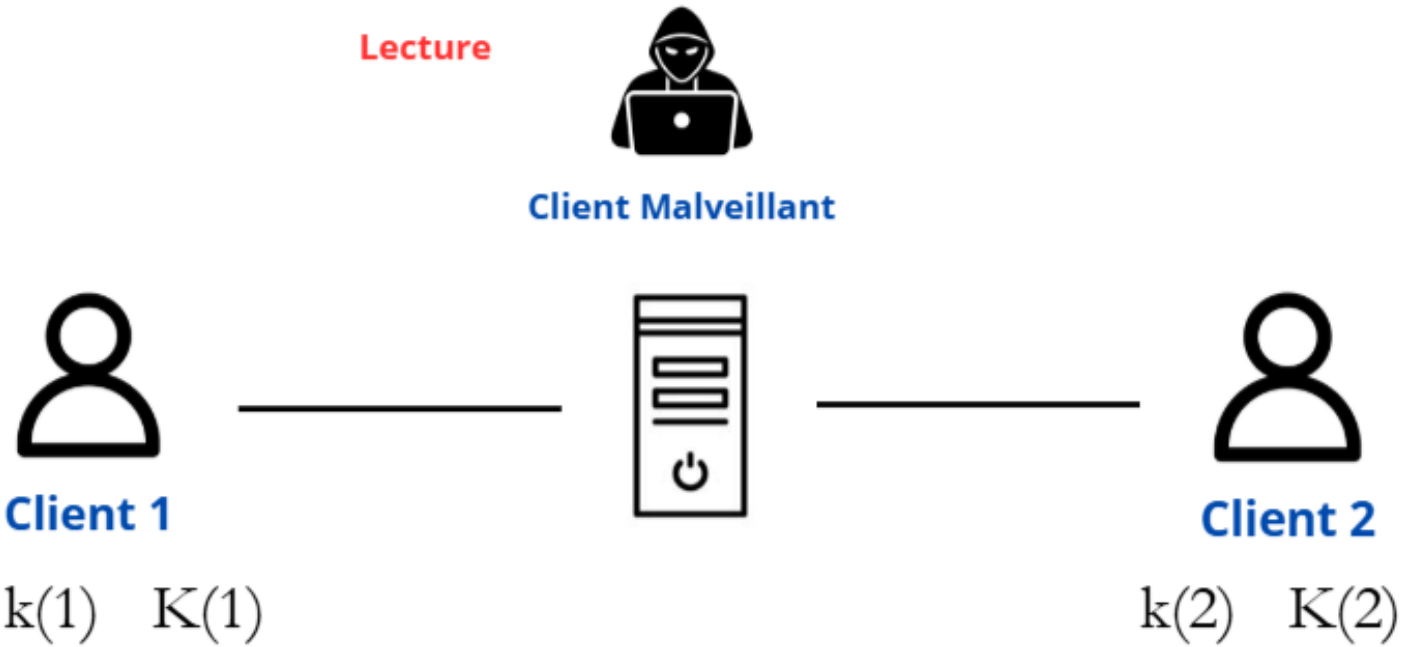


Figure: Les clients échangent leur clé publique K respective

Protocole du modèle n°1

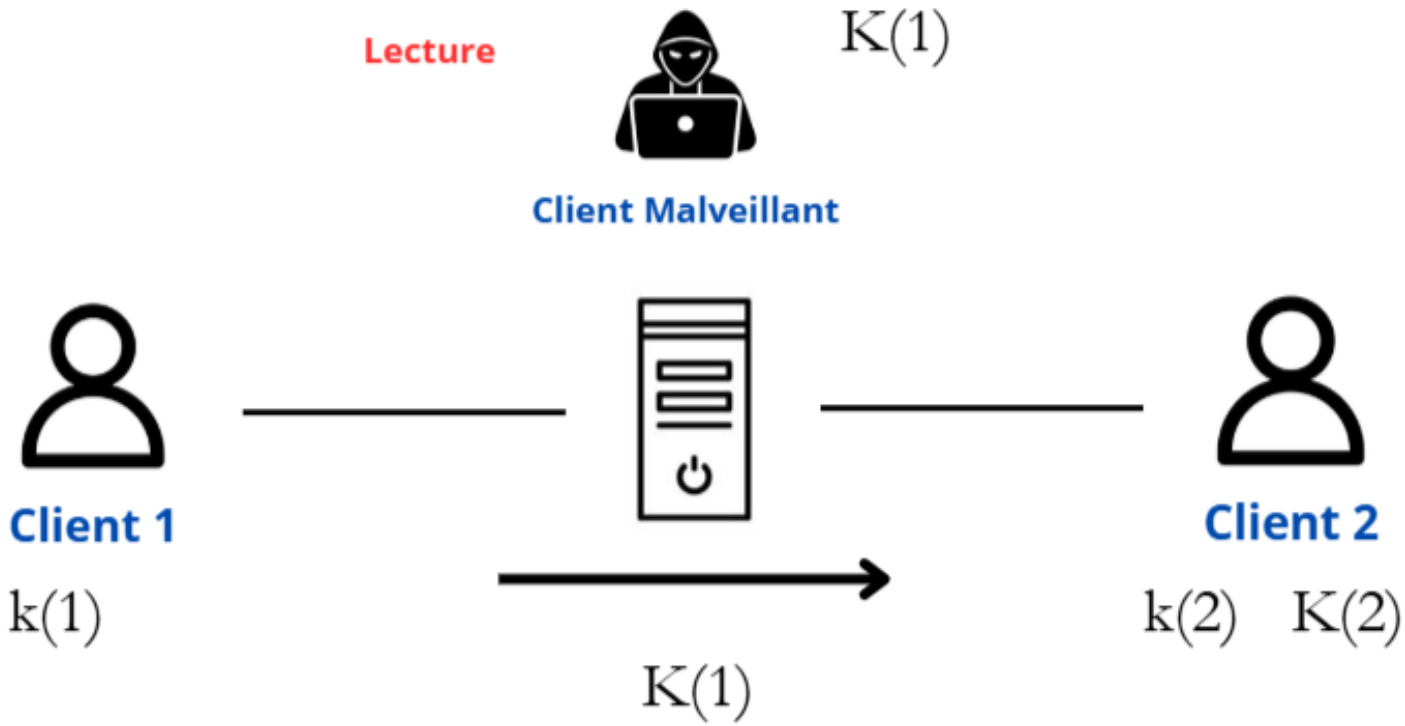


Figure: Le client 1 envoie sa clé publique $K(1)$ au client 2 en passant par le serveur

Protocole du modèle n°1

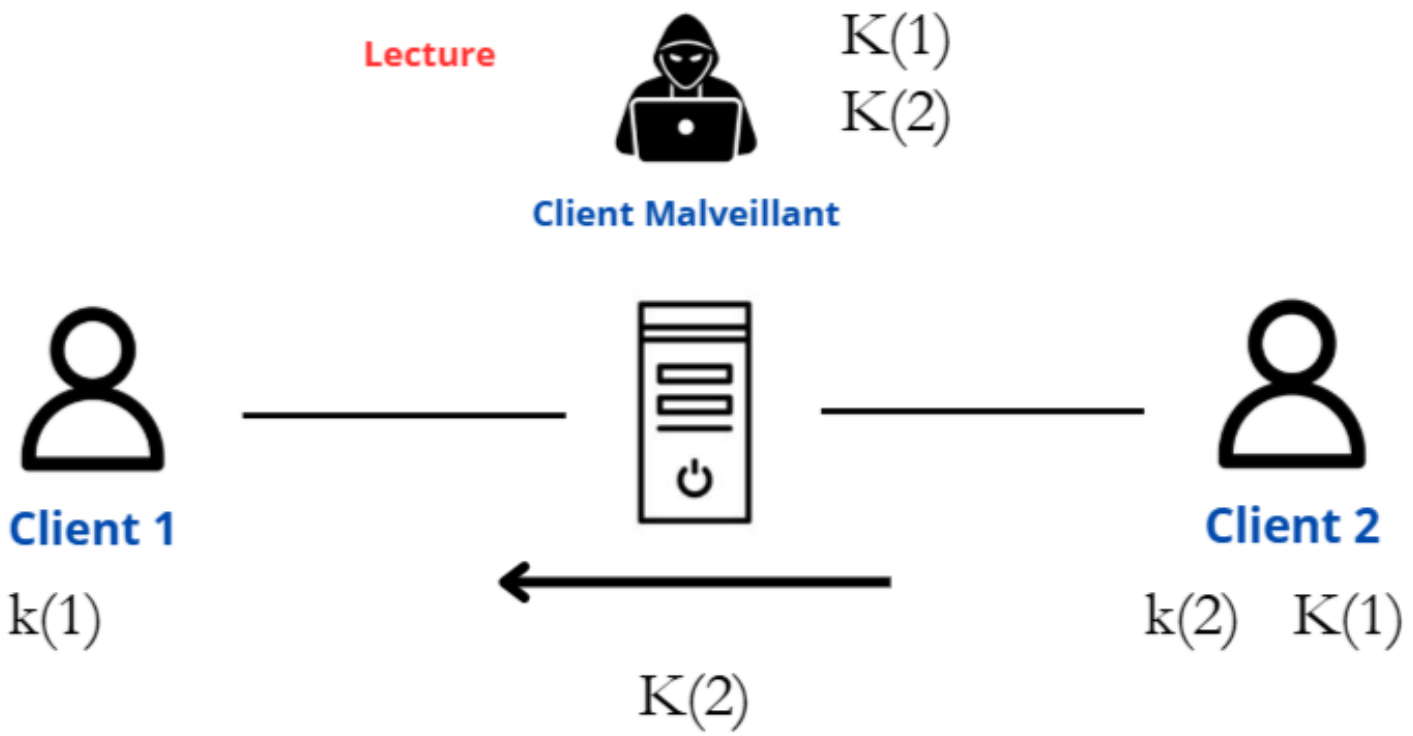


Figure: Le client 2 envoie en retour sa clé publique $K(2)$ au client 1 en passant par le serveur

Protocole du modèle n°1

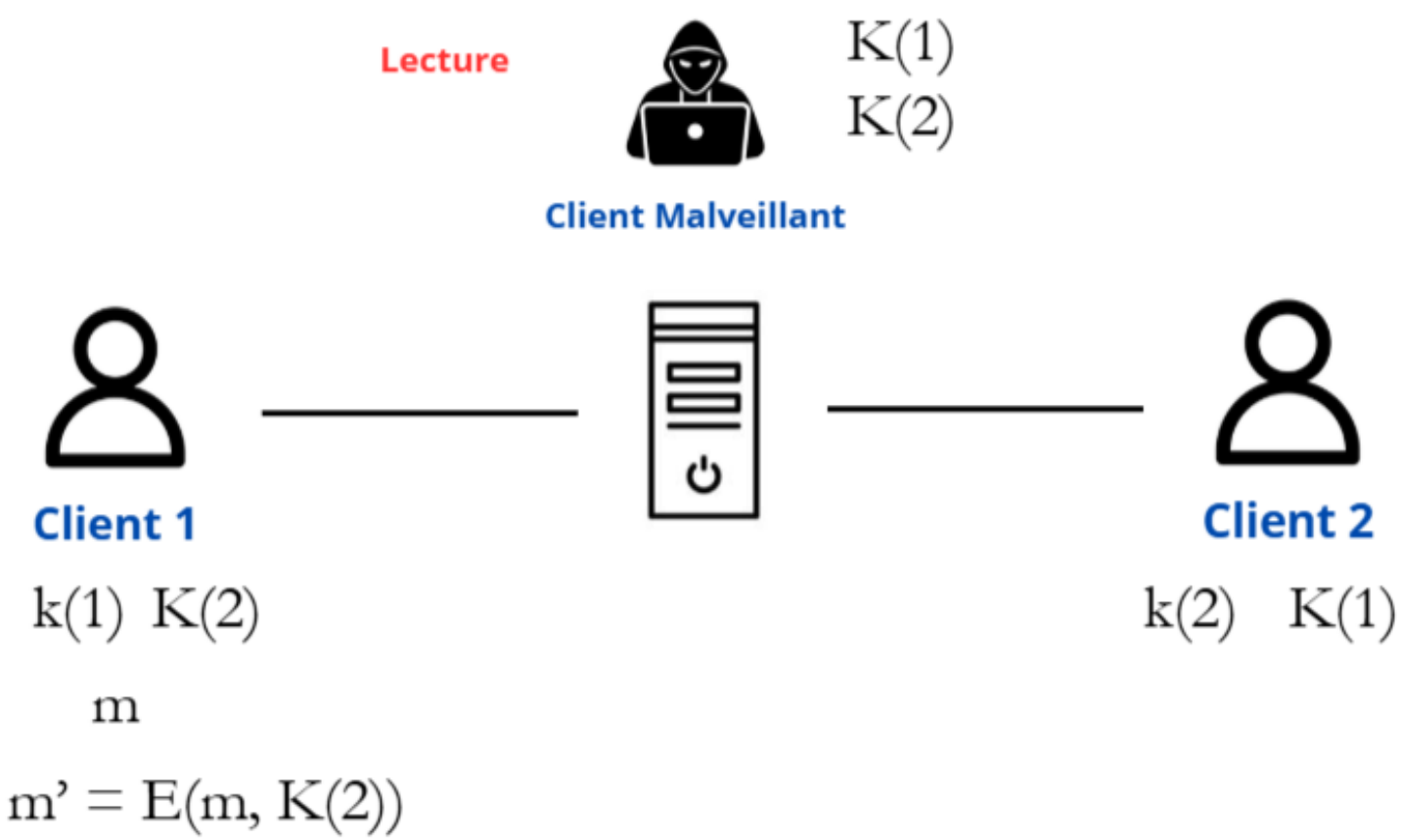


Figure: Le client 1 chiffre le message m avec la clé $K(2)$ et la fonction de chiffrement E

Protocole du modèle n°1

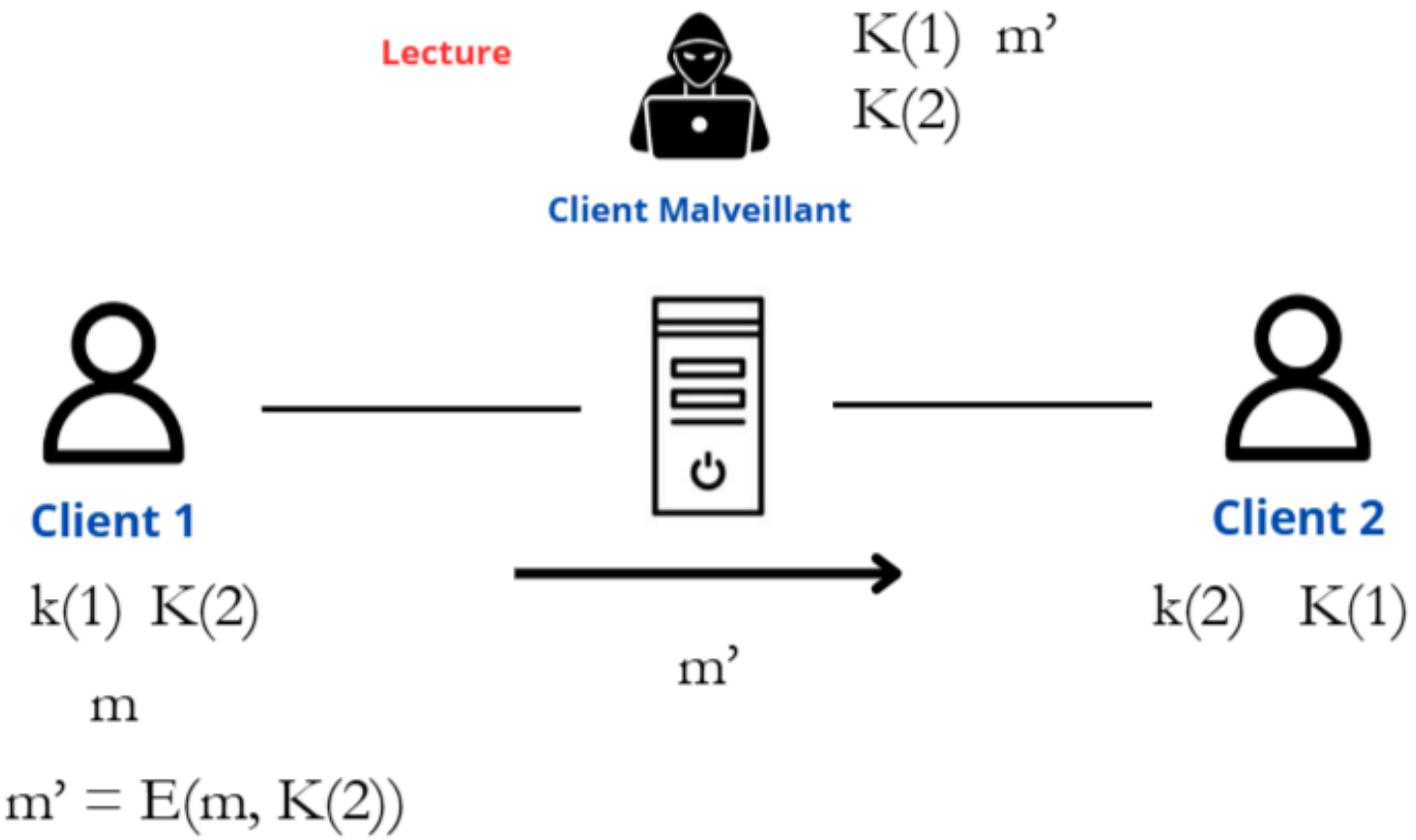


Figure: Le client 1 envoie le message chiffré m' au client 2 en passant par le serveur

Protocole du modèle n°1

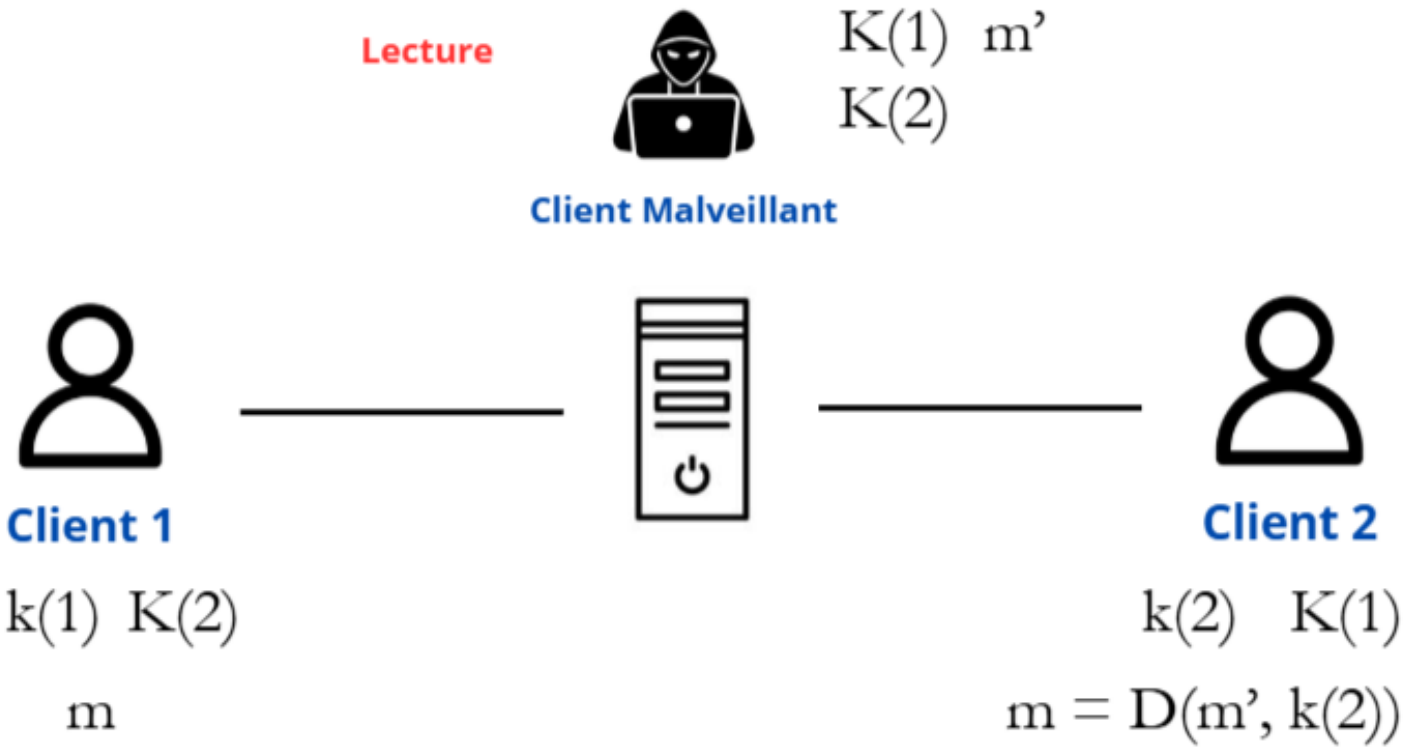


Figure: Le client 2 reçoit m' et le déchiffre avec sa clé privée $k(2)$ et la fonction de déchiffrement D

Test pratique

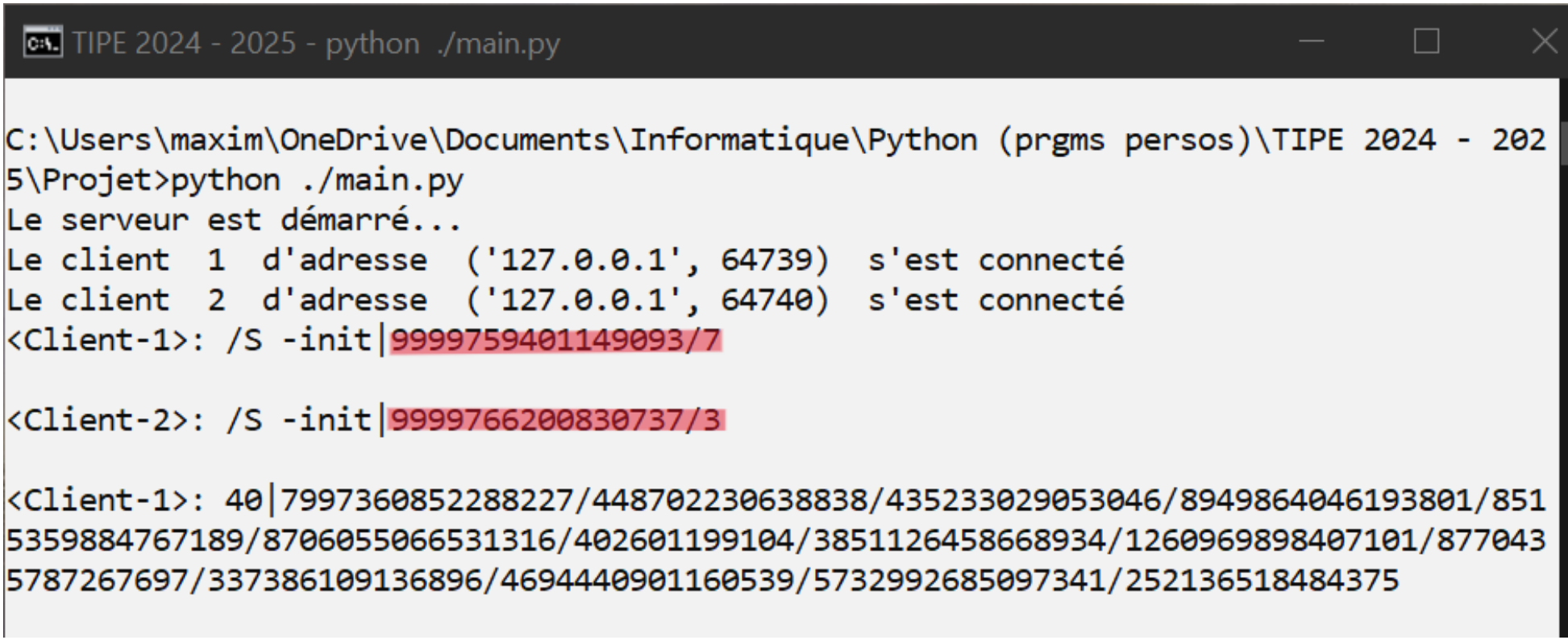


Figure: Visuel du transit du message chiffré 'Message secret ecrit par Maxime LE BLAN' par le serveur, et de ce que peut voir le client malveillant

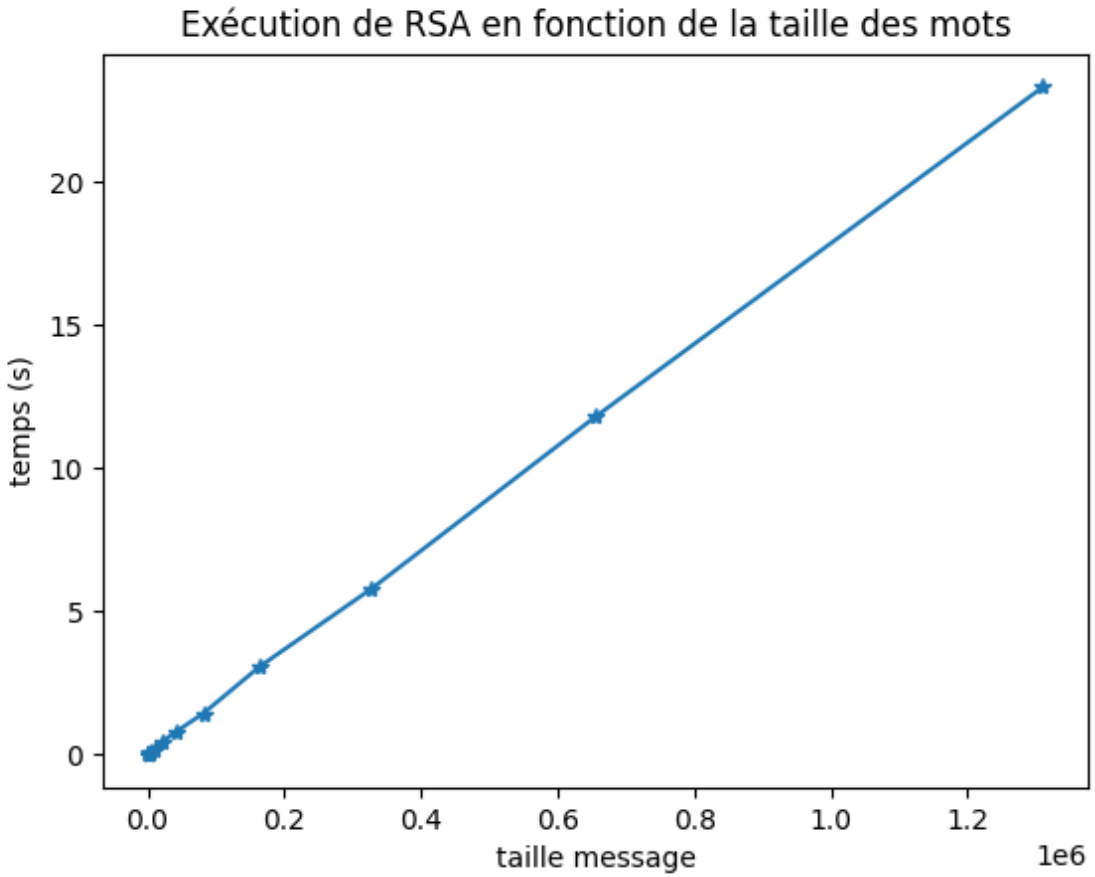
Test pratique

Taille message	temps (s)
10	0,00025
20	0,00034
40	0,00066
80	0,00133
160	0,00318
320	0,00591
640	0,01081
1280	0,02196
2560	0,04498
5120	0,0824
10240	0,17847
20480	0,3945
40960	0,76235
81920	1,40193
163840	2,83756
327680	5,64771
655360	11,34546
1310720	22,83059

Figure: Temps écoulé entre l'envoi et la réception d'un message en fonction de la taille de celui-ci avec RSA

Efficacité de l'algorithme RSA

Remarque : La complexité temporelle de l'algorithme RSA est en $O(m)$ avec m l'équivalent entier du message littéral

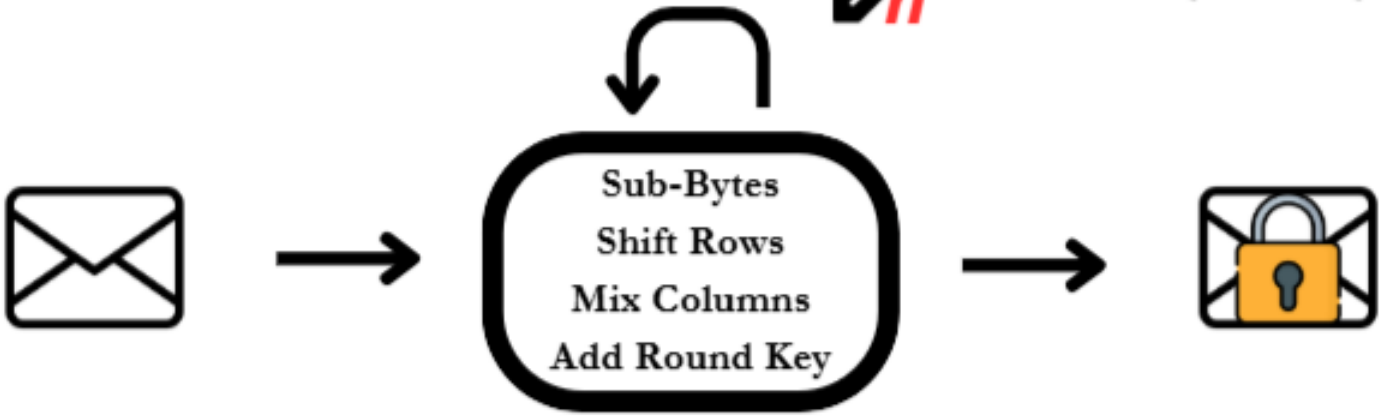


Algorithme AES

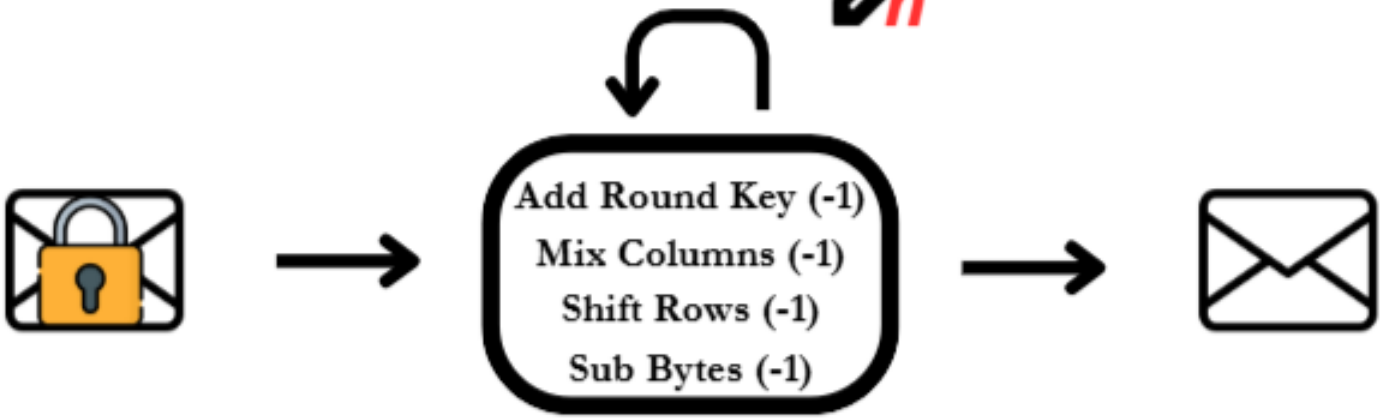


128 / 192 / 256 bits $N \in \{10; 12; 14\}$

Chiffrement : Tour n  $1 \leqslant n \leqslant N$



Déchiffrement : Tour n 



Exemple avec Sub-Bytes et Shift Rows

ShiftRows :

```
"C:\Users\maxim\OneDrive\D
9B  01  8D  F6
74  B4  AA  4B
EF  6E  5A  F1
2C  45  92  EF

9B  01  8D  F6
4B  74  B4  AA
5A  F1  EF  6E
45  92  EF  2C
```

Sub Bytes:

```
"C:\Users\maxim\OneDrive\Documents\
paquet :  9B
paquet <- GF(paquet) :  88
paquet <- int(paquet) :  136
paquet <- bin(paquet) :  10001000
paquet <- f(paquet) :  10110001
paquet <- hex(paquet) :  b1
```

```
"C:\Users\maxim\OneDrive\
9B  01  8D  F6
74  B4  AA  4B
EF  6E  5A  F1
2C  45  92  EF

b1  37  25  d4
d9  28  3a  cb
d  9  1b  ea
e7  16  4  d
```

Protocole du modèle n°2

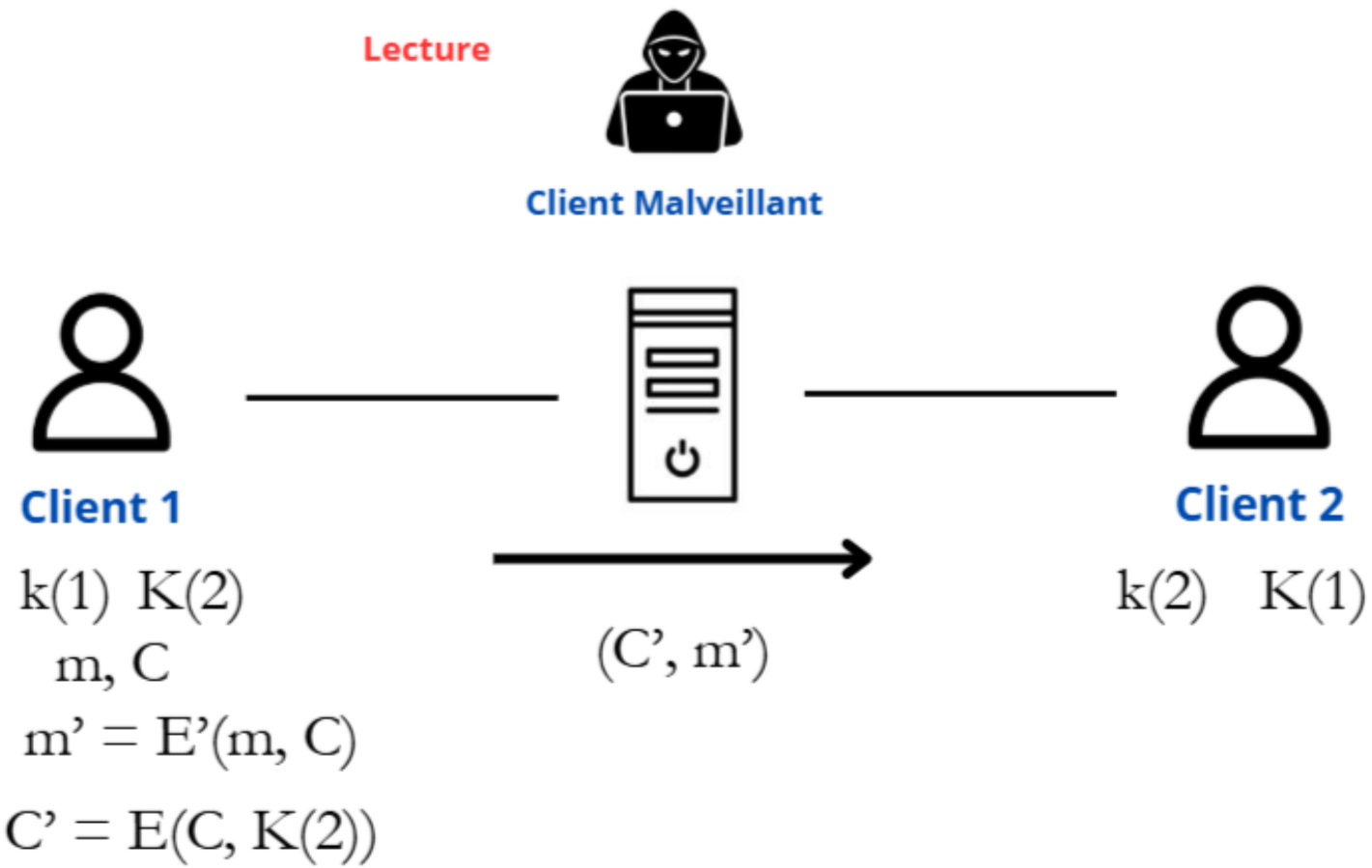
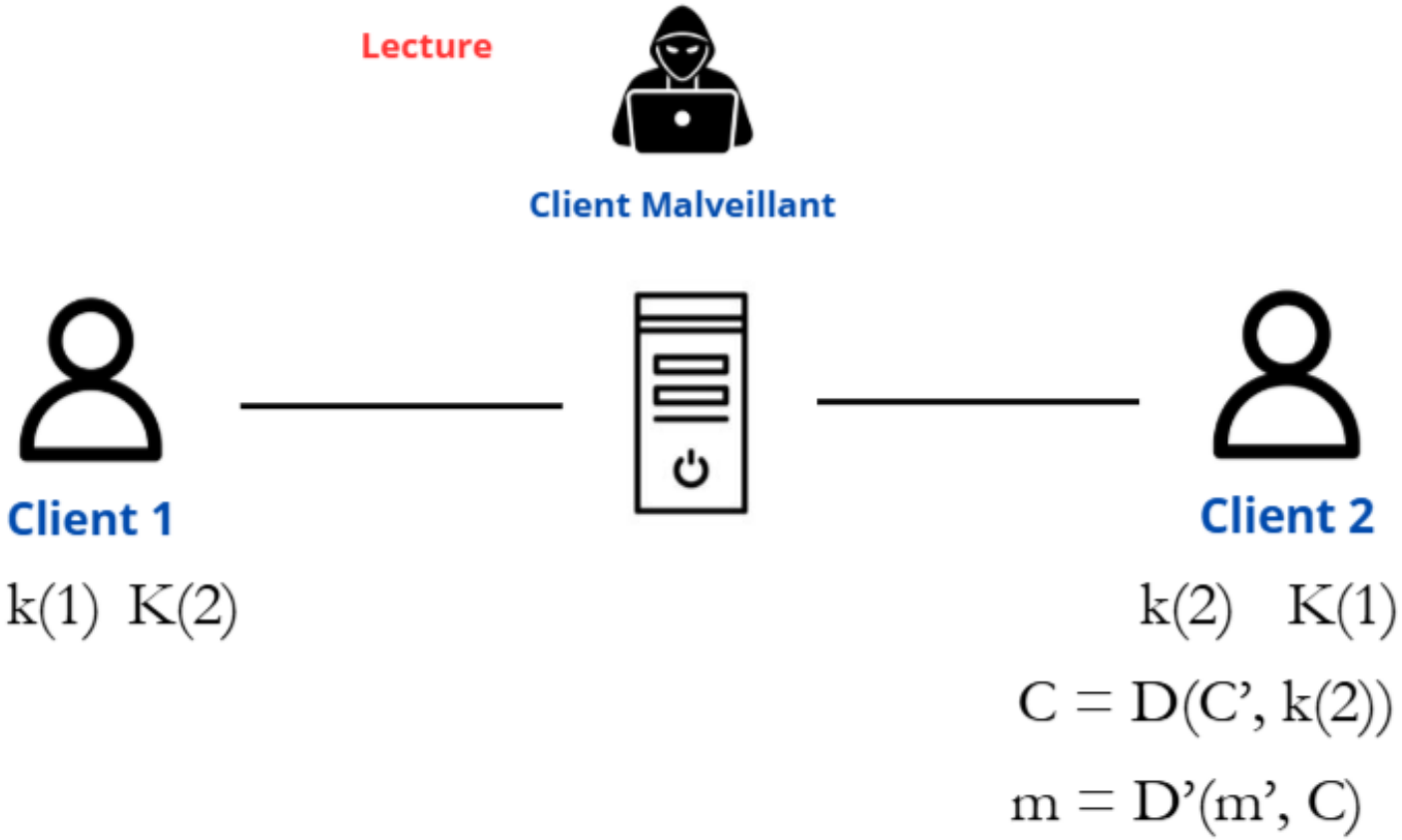


Figure: Cette fois-ci, m est chiffré avec la clé C de AES puis C est chiffrée avec $K(2)$

Protocole du modèle n°2



Test pratique

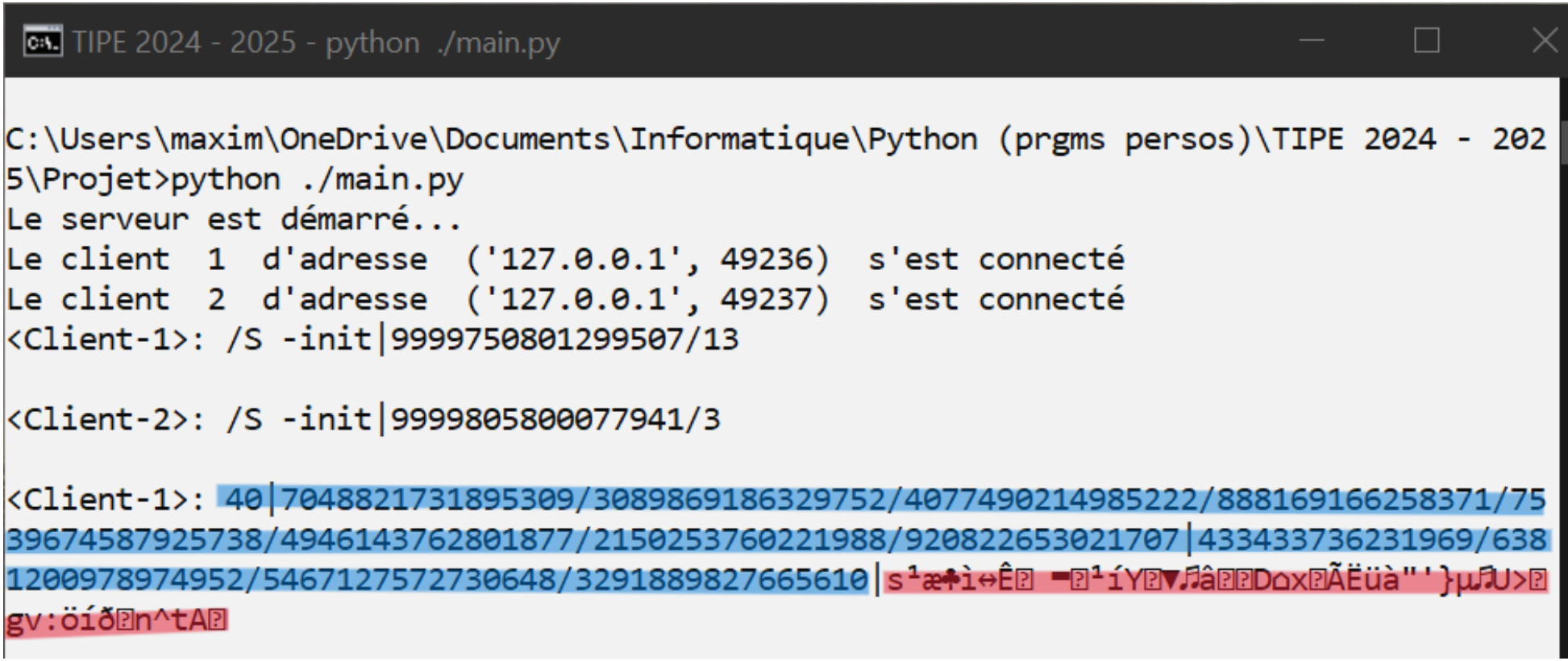


Figure: Visuel du transit du message chiffré 'Message secret ecrit par Maxime LE BLAN' par le serveur, et de ce que peut voir le client malveillant

Test pratique

AES + RSA

Taille message	temps (s)
10	0,00058
20	0,00061
40	0,00059
80	0,00059
160	0,00059
320	0,00059
640	0,0006
1280	0,0006
2560	0,0006
5120	0,00063
10240	0,00064
20480	0,00069
40960	0,00076
81920	0,00093
163840	0,00128
327680	0,00204
655360	0,00439
1310720	0,00783

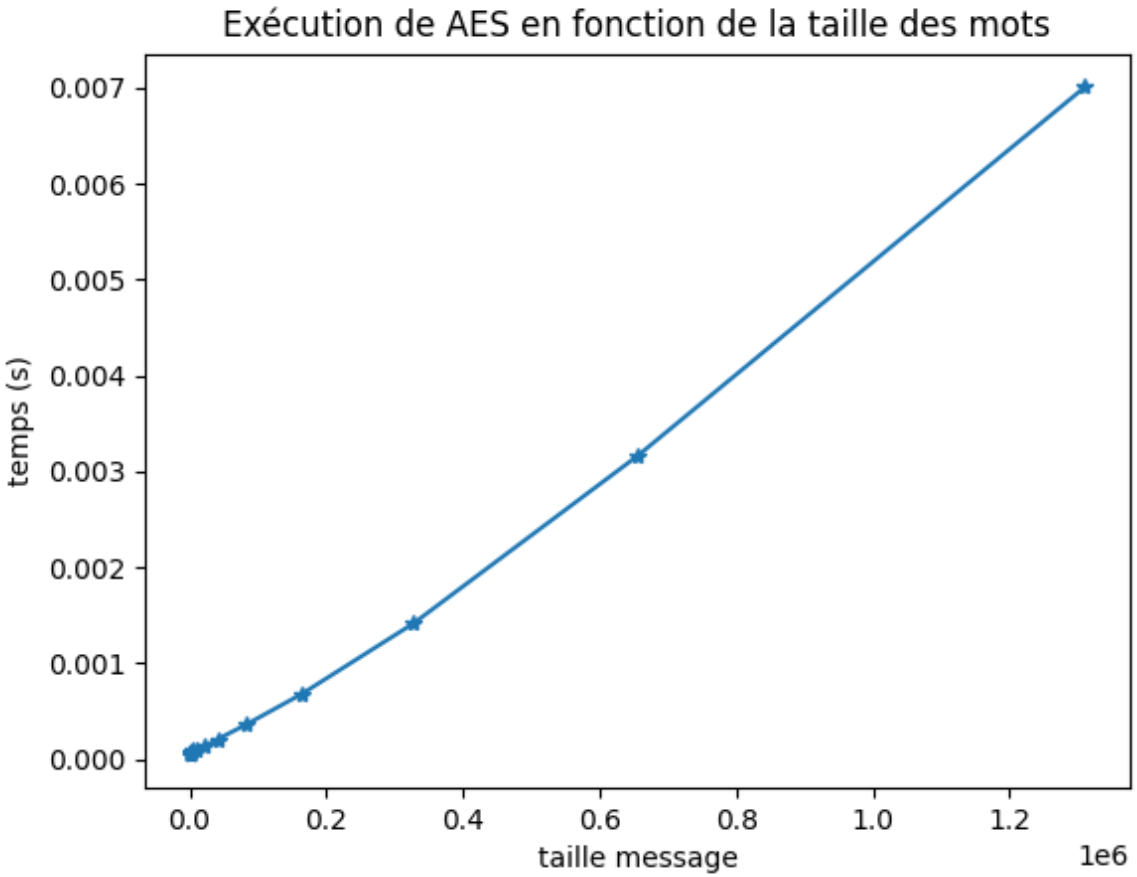
RSA

Taille message	temps (s)
10	0,00025
20	0,00034
40	0,00066
80	0,00133
160	0,00318
320	0,00591
640	0,01081
1280	0,02196
2560	0,04498
5120	0,0824
10240	0,17847
20480	0,3945
40960	0,76235
81920	1,40193
163840	2,83756
327680	5,64771
655360	11,34546
1310720	22,83059

Figure: Temps écoulé entre l'envoi et la réception d'un message en fonction de la taille de celui-ci

Efficacité de l'algorithme AES

Remarque : La complexité temporelle de l'algorithme AES est en $O(m)$ avec m l'équivalent entier du message littéral, mais AES reste en pratique plus efficace que RSA



Problème de l'Homme du milieux

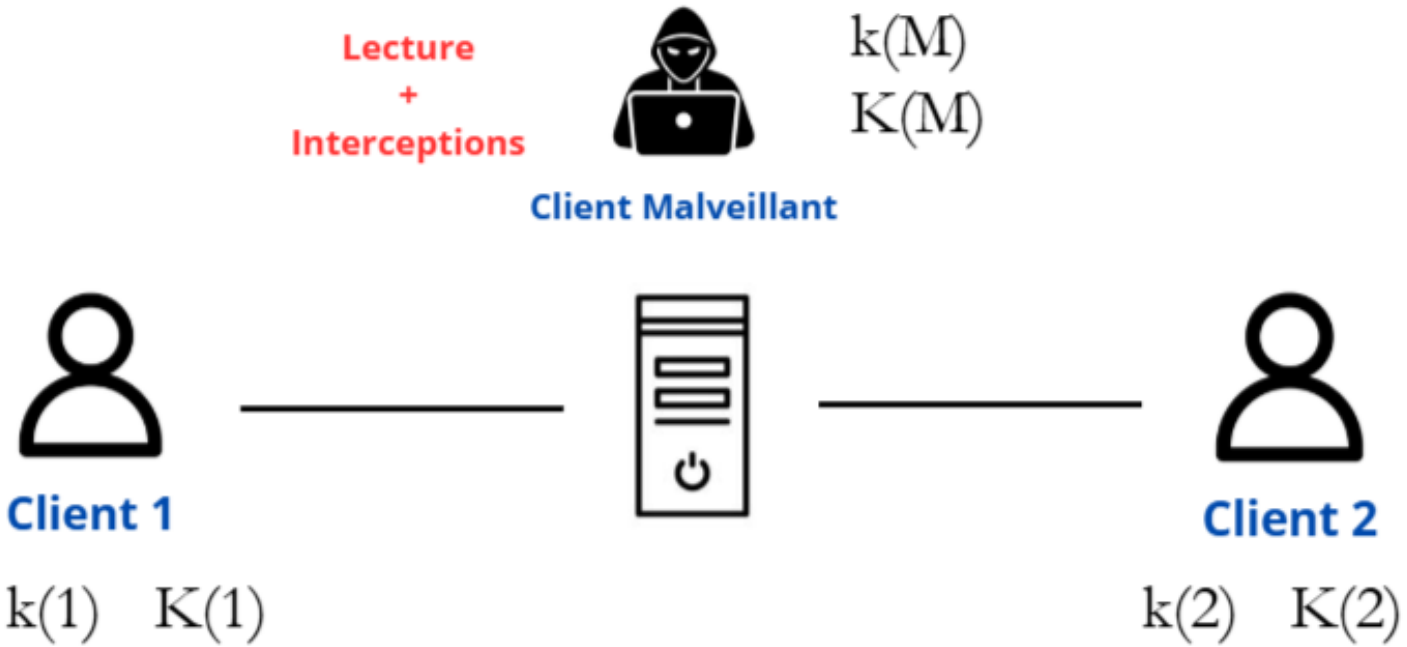


Figure: Le client malveillant crée ses propres clés $k(M)$ et $K(M)$

Problème de l'Homme du milieux

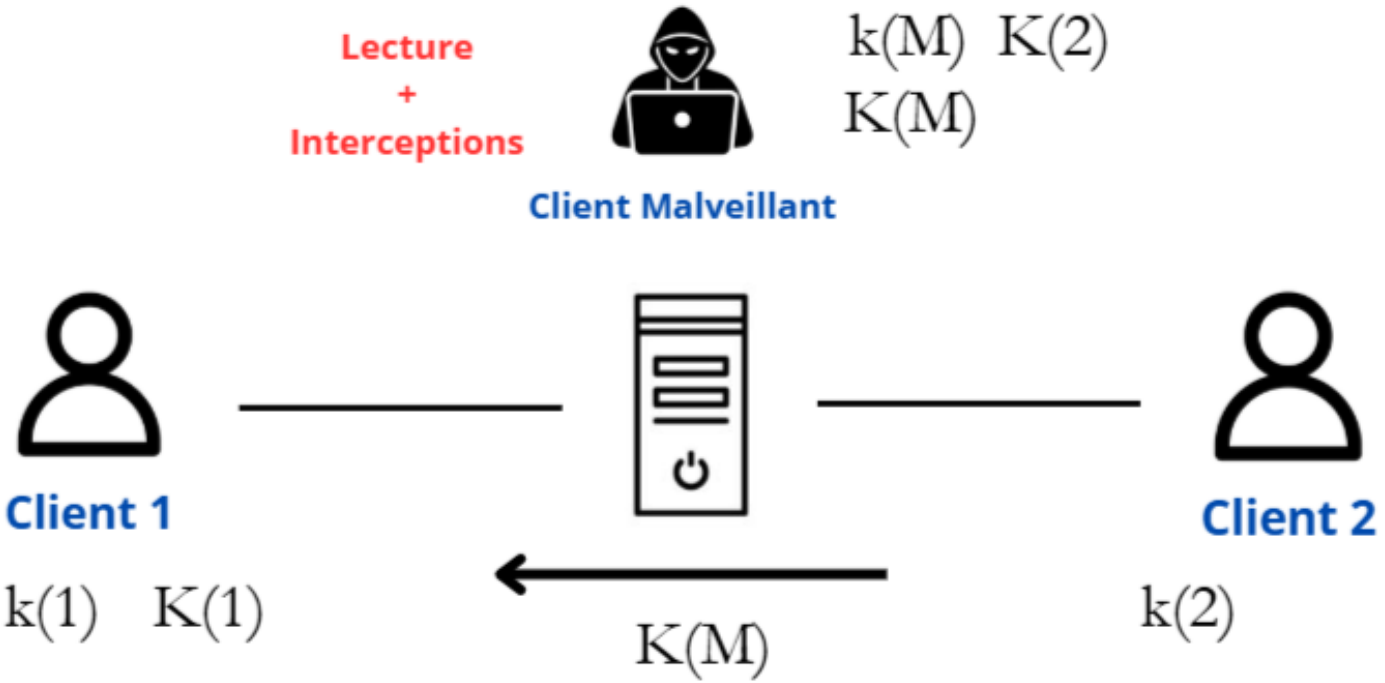


Figure: Puis il échange $K(2)$ avec $K(M)$ lors de l'envoi de $K(2)$

Problème de l'Homme du milieux

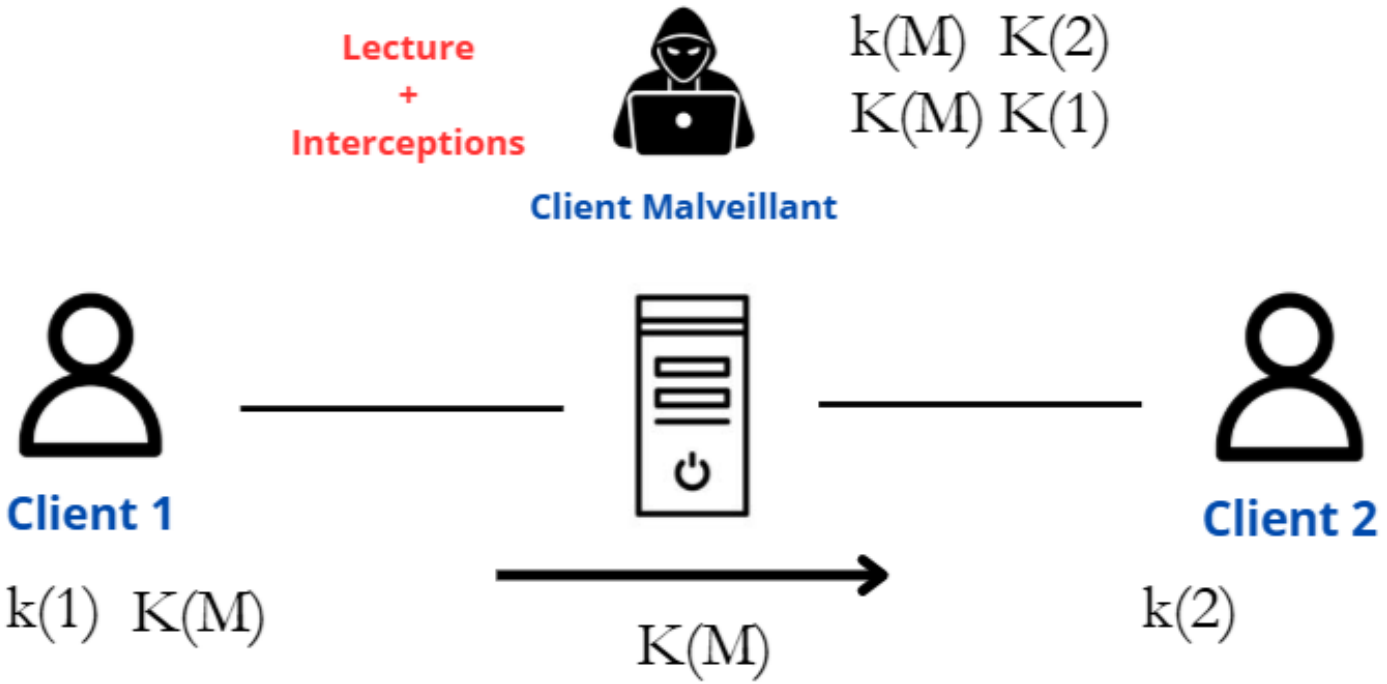


Figure: Et il échange $K(1)$ avec $K(M)$ lors de l'envoi de $K(1)$

Problème de l'Homme du milieux

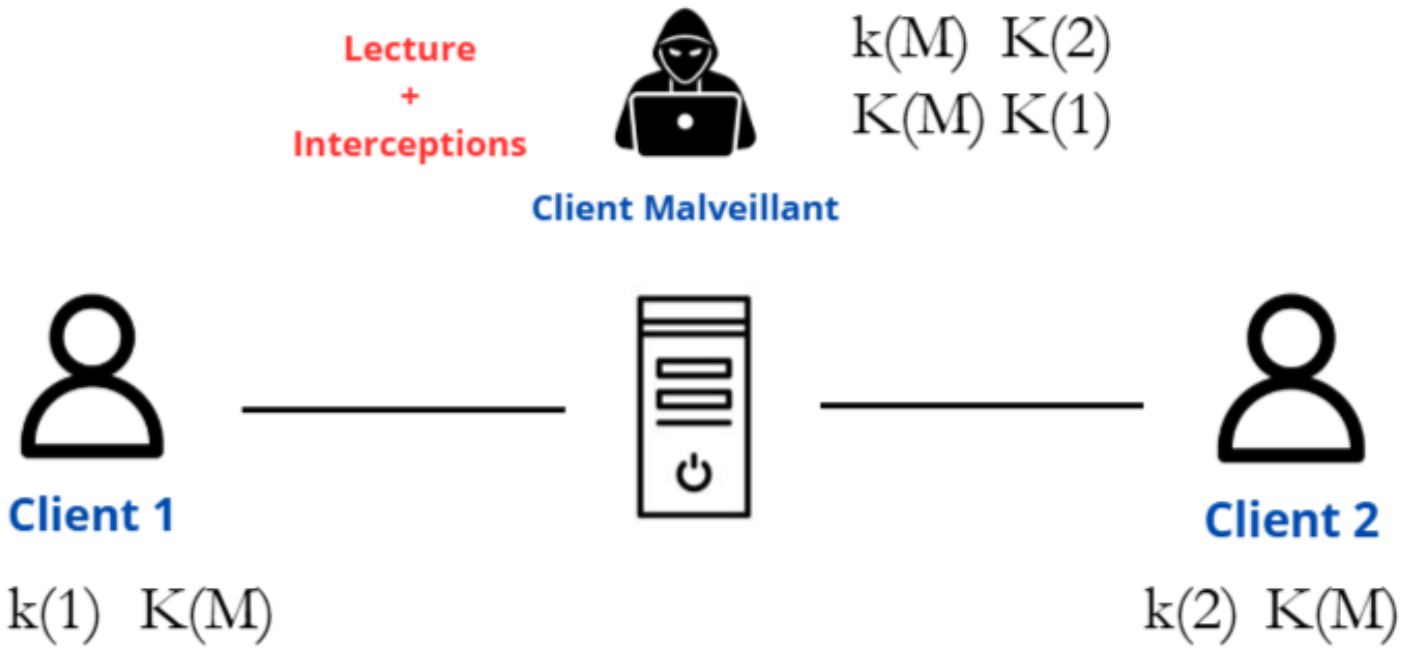
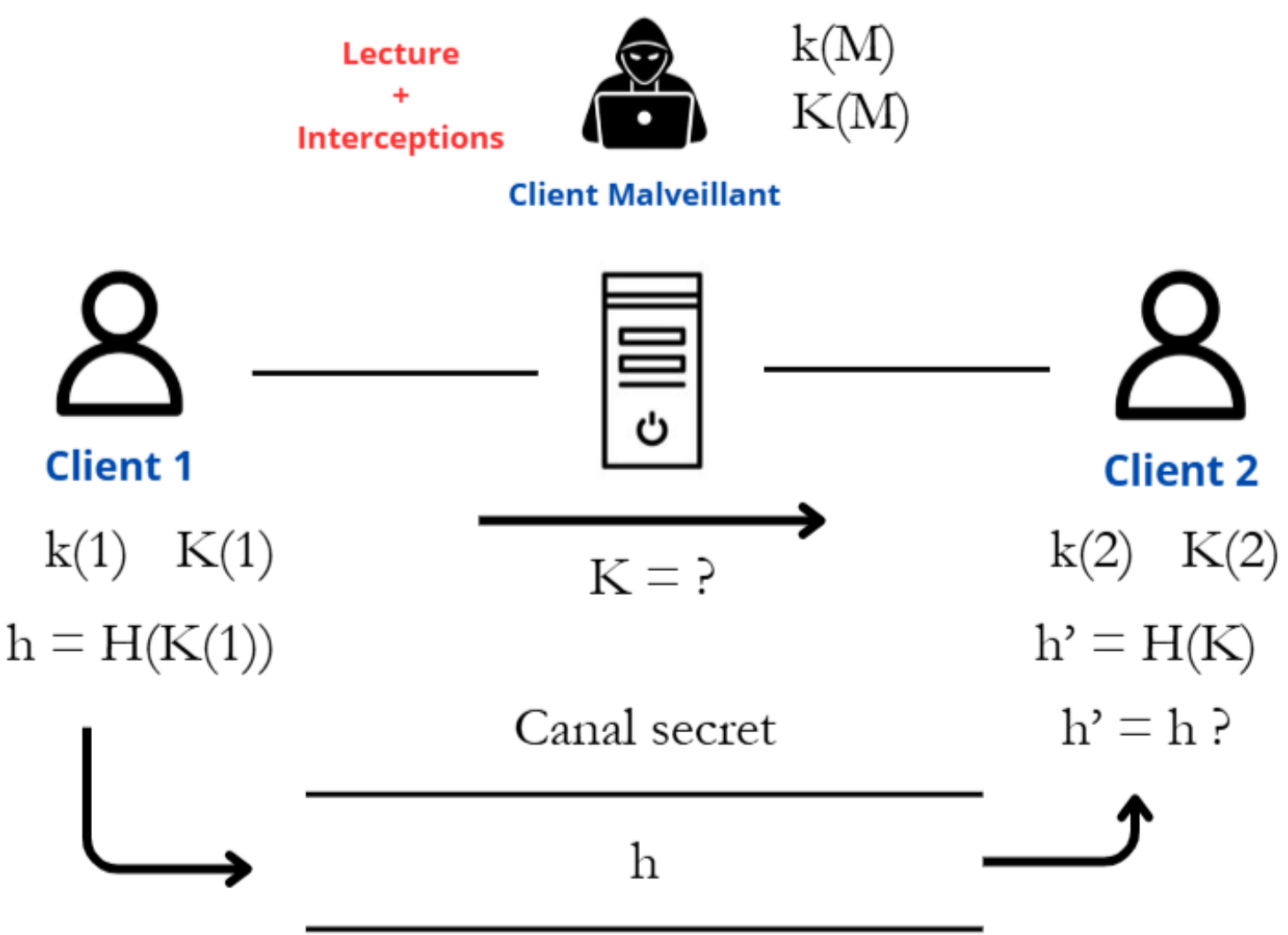


Figure: Le client malveillant peut donc lire tous les messages échangés

Protocole SAS (Short Authenticated Strings)



Test pratique

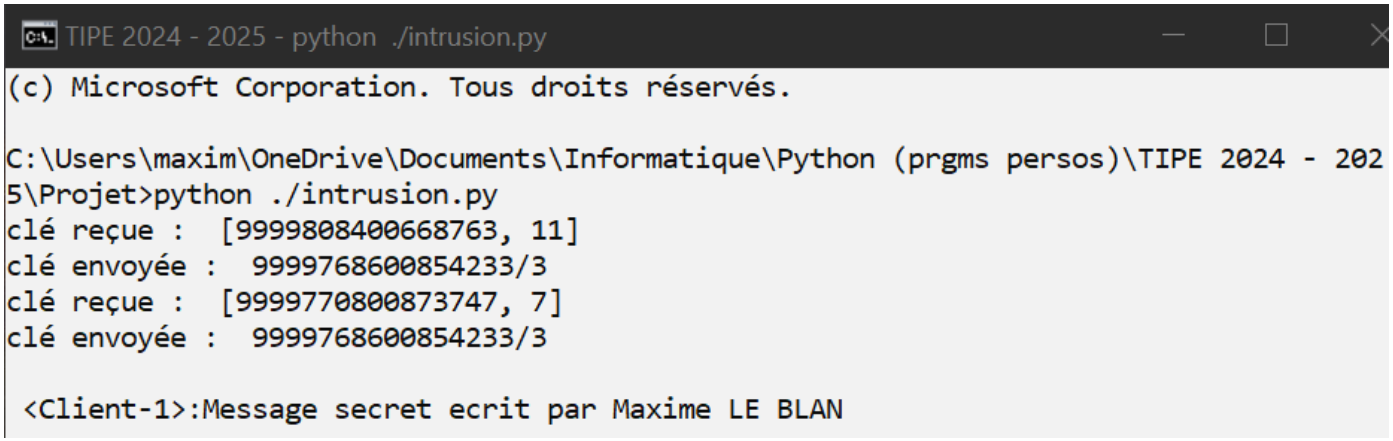


Figure: Interface du client malveillant après interception des clés

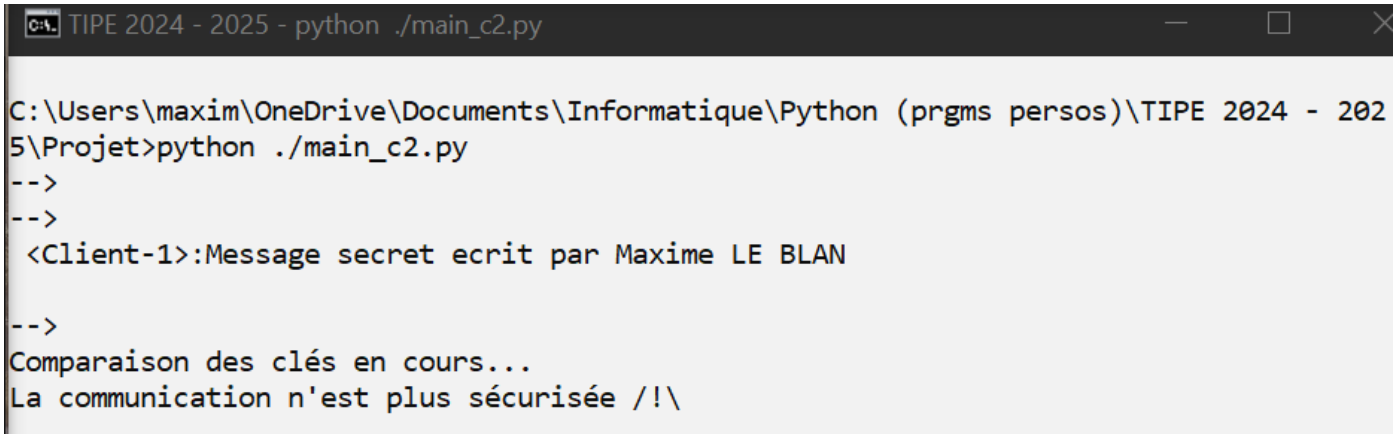


Figure: Utilisation du canal secret par le client 1

Conclusion

- Implémentation parallèlisée d'un réseau
- Implémentation d'un modèle efficace résistant à la lecture intrusive des données

Problèmes :

- Implémentation du canal secret peu sécurisée et très simplifiée
- L'intrusion du client malveillant dans la communication bloque tout échange sécurisé de données

Pistes de réflexion :

- Chercher une alternative au protocole SAS : utilisation de certificats pour l'authentification

Annexe - Preuve déchiffrement RSA

- Soient $c, m \in \mathbb{N}$, $p, q \in \mathbb{P}$ $(p \neq q)$ et $e, d \in \mathbb{N}$ tel que $ed \equiv 1 \pmod{(p-1)(q-1)}$. Posons $n = p \times q$
- Comme $c \equiv m^e \pmod{n}$, alors $c^d \equiv m^{ed} \pmod{n}$
- De plus, $ed \equiv 1 \pmod{(p-1)(q-1)}$
  $\Rightarrow \exists k \in \mathbb{N}, ed = 1 + k(p-1)(q-1)$
  Donc $c^d \equiv m^{1+k(p-1)(q-1)} \pmod{n}$

Annexe - Preuve déchiffrement RSA

- Par conséquent :
  - Si m n'est pas un multiple de p, d'après le Petit théorème de Fermat,
    $m^{p-1} \equiv 1 \pmod{p}$
    $\Rightarrow m^{ed} \equiv m^{1+k(p-1)(q-1)} \equiv m \times (m^{p-1})^{k(q-1)} \pmod{p}$
    $\Rightarrow m^{ed} \equiv m \pmod{p}$
  - Si m est un multiple de p, alors $m^{ed} \equiv 0 \pmod{p}$
    $\Rightarrow m^{ed} \equiv m \pmod{p}$

  Donc $m^{ed} \equiv m \pmod{p}$ et de même, $m^{ed} \equiv m \pmod{q}$
- p et q étant premiers, on a donc :
  $m^{ed} - m \equiv 0 \pmod{pq}$
  $\Rightarrow m^{ed} \equiv m \pmod{n}$
  $\Rightarrow c^d \equiv m \pmod{n}$

Annexe - Fonction de hachage idéale

Propriétés :

- déterministe
- fonction de hachage avec une complexité amortie en $O(1)$
- résistance à la préimage : impossibilité, pour une valeur de hachage donnée, de construire un message ayant cette valeur
- résistance à la seconde préimage : impossibilité de trouver un second message ayant la même valeur de hachage qu'un message donné
- résistance aux collisions
- modifier légèrement un message modifie considérablement la valeur de hachage

Annexe - Modules utilisés

```
1   # Imoportation de modules auxiliaires de Python
2   import random as rd
3   import time
4   import threading
5   import socket
6   import sys
7   # Module utilisé pour l'utilisation d'une fonction de hachage
8   import hashlib
9   # Modules nécessaires pour l'implémentation d'AES avec des bibliothèques
10  from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
11  from cryptography.hazmat.backends import default_backend
12  from cryptography.hazmat.primitives import padding
13  # Importation d'autres fichiers Python
14  import fonctions_utiles as fu
15  import algos_chiffrement as algos
```

Annexe - algos_chiffrement.py

```python
def aux_modulo(x, n, m, acc_reste):
    """ Calcul de x^n modulo m avec la méthode d'exponentiation rapide"""
    if n == 1:
        return acc_reste % m
    elif n % 2 == 0:
        return aux_modulo(x*x % m, int(n/2), m, acc_reste*x % m)
    else:
        return aux_modulo(x*x % m, int((n-1)/2), m, acc_reste*x*x % m)

def modulo_exp(x:int, n:int, m:int):
    """ Fonction qui calcule x^n modulo m """
    if n == 0:
        return 1 % m
    else:
        return aux_modulo(x, n, m, x)

def string_to_int(chaine: str, base=27):
    """ Fonction qui convertit une chaîne de caractères en entier et renvoie cet
        entier """
    n = len(chaine)
    entier = 0

    for k in range(1, n+1):
        entier += dico_char_a_int[chaine[k-1]] * base**(n - k)
    return entier
```

Annexe - algos_chiffrement.py

```python
def int_to_string(entier: int, taille_mot: int, b=27):
    """ Fonction qui convertit un entier en chaîne de caractères et renvoie cette
        chaîne """

    quotient = 0
    diviseur = 0
    dividende = entier
    mot = ""

    for i in range(taille_mot - 1, 0, -1):
        diviseur = b**i
        quotient = dividende // diviseur
        mot += liste_int_a_char[quotient]
        dividende -= quotient * diviseur
    mot += liste_int_a_char[dividende%b]

    return mot

# Création de l'exception "InvNotFound" utile dans la suite
class InvNotFound(Exception):
    pass
```

Annexe - algos_chiffrement.py

```python
def inverse_mod(b: int, n: int):
    """ Fonction qui prend en paramètre deux entiers b et n et renvoie l'inverse de
        b modulo n (ie b^(-1))
    ou bien renvoie l'erreur 'InvNotFound' si b^(-1) n'existe pas """
    n_0 = n
    b_0 = b
    t_0 = 0
    t = 1
    q = int(n_0/b_0)
    r = n_0 - q * b_0
    while r > 0:
        temp = t_0 - q * t
        if temp >= 0:
            temp = temp % n
        else:
            temp = n - ((-temp) % n)
        t_0 = t
        t = temp
        n_0 = b_0
        b_0 = r
        q = int(n_0/b_0)
        r = n_0 - q * b_0
    if b_0 != 1:
        raise InvNotFound()
    else:
        return t
```

Annexe - algos_chiffrement.py

```python
def creation_cles(lst_nb_prem: list):
    """ Fonction prenant en paramètre une liste de nombres premiers et qui crée
        puis renvoie un couple de la forme
     (cle_publique, cle_privee) en suivant la méthode de création des clés de l'
        algorithme de chiffrement RSA\n
     - cle_publique : tuple int \n
     - cle_privee : int """
    e = 2
    # On choisit aléatoirement les valeurs de p et q distinctes
    p = lst_nb_prem.pop(rd.randint(0, len(lst_nb_prem) - 1))
    q = rd.choice(lst_nb_prem)
    # On choisit e et phi_n
    n = p * q
    phi_n = (p - 1) * (q - 1)

    for i in range(2, phi_n): # on cherche le premier e qui convient en partant de
        e = 2
        if fu.pgcd(i, phi_n) == 1:
            e = i
            break
    d = inverse_mod(e, phi_n)

    return ((n, e), d) # on renvoie (clé publique, clé privée)
```

Annexe - algos_chiffrement.py

```python
def chiffrer_RSA_naif(m: int, cle_pb: tuple):
    """ Fonction qui prend en paramètre un entier m qui correspond au message à chiffrer et un tuple cle_pb qui
    correspond à la clé de chiffrement. Elle renvoie un entier correspondant au message chiffré """

    assert (m < cle_pb[0])
    # print("début calcul chiffrement et cle_pb : ", cle_pb)
    reste = 1
    for _ in range(cle_pb[1]):
        reste *= m
        reste = reste % cle_pb[0]
    return reste

def chiffrer_RSA_v2(m: int, cle_pb: tuple):
    """ Fonction qui prend en paramètre un entier m qui correspond au message à chiffrer et un tuple cle_pb qui
    correspond à la clé de chiffrement. Elle renvoie un entier correspondant au message chiffré """

    assert (m < cle_pb[0])
    return modulo_exp(m, cle_pb[1], cle_pb[0])
```

Annexe - algos_chiffrement.py

```python
def dechiffrer_RSA_naif(x: int, cle_pv: int, n: int):
    """ Fonction prenant en paramètre un entier x correspondant au message à dé
        chiffrer, cle_pv un entier
    correspondant à la clé de déchiffrement et n issue de la cle de chiffrement du
        message. Elle renvoie un entier
    correspondant au message déchiffré """

    reste = 1
    for _ in range(cle_pv):
        reste *= x
        reste = reste % n

    return reste

def dechiffrer_RSA_v2(x: int, cle_pv: int, n: int):
    """ Fonction prenant en paramètre un entier x correspondant au message à dé
        chiffrer, cle_pv un entier
    correspondant à la clé de déchiffrement et n issue de la cle de chiffrement du
        message. Elle renvoie un entier
    correspondant au message déchiffré """

    return modulo_exp(x, cle_pv, n)
```

Annexe - algos_chiffrement.py

```python
def chiffrer_chaine_RSA(chaine:str, cle_pb:tuple, taille_facteur=-1):
    """ Fonction qui prend en paramètre une chaine de caractères 'chaine' et un
        tuple 'cle_pb' et renvoie
    une liste d'entiers correspondants aux facteurs de taille 'taille_facteur',
        chiffrés avec la fonction 'chiffrer_RSA(...)',
    dont la concaténation forme la chaine 'chaine' avant son chiffrement """
    if taille_facteur == (-1):
        taille_facteur = len(chaine)
    chaine_chiffree = []
    chaine_fractionnee = []
    chaine_courante = ""

    for i in range(len(chaine)): # On décompose 'chaine' en une liste de facteurs
        de taille 'taille_facteur'
        chaine_courante += chaine[i]
        if len(chaine_courante) % taille_facteur == 0:
            chaine_fractionnee.append(chaine_courante)
            chaine_courante = ""

    if chaine_courante != "":
        for i in range(len(chaine_courante), taille_facteur): # on ajoute des
            espaces pour que le dernier facteur soit de taille taille_facteur
            chaine_courante += "␣"
        chaine_fractionnee.append(chaine_courante) # on ajoute le dernier facteur
            de taille < 'taille_facteur' (sans compter les espaces)

    for facteur in chaine_fractionnee: # On chiffre tous les facteurs avec la
        fonction 'chiffrer_RSA(...)'
        int_facteur_courant = string_to_int(facteur, base=95)
        chaine_chiffree.append(chiffrer_RSA_v2(int_facteur_courant, cle_pb))
    return chaine_chiffree
```

Annexe - algos_chiffrement.py

```python
def dechiffrer_chaine_RSA(liste:list, cle_pv: int, n: int, taille_entiers=1):
    """ Fonction qui prend en paramètre une liste d'entiers 'liste', un entier 'cle_pv' et un entier n et
    renvoie la concaténation des chaines de caractères déchiffrées avec 'dechiffrer_RSA(...)' à partir des entiers
    contenu dans 'liste' représentant des chaines de caractères de taille 'taille_entiers' """

    chaine_dechiffree = ""

    for i in range(len(liste)): # On déchiffre tous les termes de liste sauf le dernier
        chaine_courante = int_to_string(dechiffrer_RSA_v2(liste[i], cle_pv, n), taille_entiers, b=95)
        chaine_dechiffree += chaine_courante

    return chaine_dechiffree
```

Annexe - algos_chiffrement.py

```python
def aux_bytesub(a: str):
    """ Fonction auxiliaire de bytesub qui prend en entrée un élément codé en hexad
        écimal et renvoie le résultat
    de la fonction ByteSub appliquée à celui-ci """

    # on construit b_prime = GF(a)
    m_gf_i = m_gf_hex_a_dec[a[0]]
    m_gf_j = m_gf_hex_a_dec[a[1]]
    b_prime = MATRICE_GF[m_gf_i][m_gf_j]
    # on construit b à partir de b_prime
    b_prime = int(b_prime, 16)
    b_prime = str(bin(b_prime))
    b_prime = b_prime[2:]
    b_prime = fu.remplir_zeros(b_prime, 8)
    b_prime = [[int(b_prime[i])] for i in range(len(b_prime))]
    b = fu.produit_matriciel(mat_f1, b_prime, (len(mat_f1), len(mat_f1)), (len(
        b_prime), 1))
    b = fu.somme_matricielle(b, mat_f2, len(mat_f2), 1)
    for i in range(len(b)):
        b[i][0] = b[i][0] % 2
    b_final = ""
    for chiffre in b:
        b_final += str(chiffre[0])
    b_final = fu.remplir_zeros(b_final, 8)
    b_final = fu.bin_vers_dec(b_final)
    b_final = str(hex(b_final))

    return b_final[2:]
```

Annexe - algos_chiffrement.py

```python
def bytesub(matrice: list[list]):
    """ Fonction bytesub qui prend en entrée une matrice codée en hexadécimal et renvoie le résultat
    de la fonction aux_bytesub de AES appliquée à celle-ci """

    return [[aux_bytesub(matrice[i][j]) for j in range(len(matrice))] for i in range(len(matrice))]

def shiftrows(matrice: list[list]):
    """ Fonction qui applique la fonction ShiftRows de AES sur la matrice """
    n = len(matrice)
    matrice_finale = []
    for d in range(n):
        matrice_finale.append(fu.decale_liste(matrice[d], d))

    return matrice_finale
```

Annexe - algos_chiffrement.py

```python
# Algorithme AES implémenté avec une bibliothèque
def chiffrer_AES(message:str, cle:bytes, vecteur:bytes):
    """ Fonction qui prend en paramètre un message, une clé et un vecteur et
        renvoie une chaine de
    caractère correspondant au chiffrement de 'message' avec l'algorithme AES en
        utilisant la clé 'cle'
    et le vecteur d'initialisation 'vecteur'\n
    - message : chaine de caractères\n
    - cle : chaine de caractères sur 32 octets\n
    - vecteur : chaine de caractères sur 16 octets"""

    padder = padding.PKCS7(128).padder()
    padded_data = padder.update(message.encode('utf-8'))
    padded_data += padder.finalize()

    cipher = Cipher(algorithms.AES(cle), modes.CBC(vecteur), backend=
        default_backend())
    encryptor = cipher.encryptor()
    message_chiffre = encryptor.update(padded_data) + encryptor.finalize()

    return message_chiffre
```

Annexe - algos_chiffrement.py

```python
def dechiffrer_AES(message_chiffre, cle:bytes, vecteur:bytes):
    """ Fonction qui prend en paramètre un message, une clé et un vecteur et renvoie une chaine de
    caractère correspondant au déchiffrement de 'message_chiffre' avec l'algorithme AES en utilisant la clé 'cle'
    et le vecteur d'initialisation 'vecteur'\n
    - message_chiffre : chaine de caractères\n
    - cle : chaine de caractères sur 32 octets\n
    - vecteur : chaine de caractères sur 16 octets"""

    decryptor = Cipher(algorithms.AES(cle), modes.CBC(vecteur), backend=default_backend()).decryptor()
    decrypted_data = decryptor.update(message_chiffre) + decryptor.finalize()

    unpadder = padding.PKCS7(128).unpadder()
    unpadded_data = unpadder.update(decrypted_data)
    unpadded_data += unpadder.finalize()

    return unpadded_data.decode('utf-8')
```

Annexe - serveur.py

```
1   class Serveur:
2       """ Classe permettant de simuler un serveur informatique simplifié à l'aide du
              module socket """
3
4       def __init__(self, modele:int, port=6390):
5           self.host = ""
6           self.port = port
7           self.socket = None
8           self.clients = []
9           self.threads = {}
10          self.id_client = 1
11          self.arret = False
12          self.modele = modele
13
14      def close(self):
15          """ Fonction qui vérifie si le serveur doit s'éteindre ou non """
16          while True:
17              if self.arret == True:
18                  break
```

Annexe - serveur.py

```
1       def connections(self):
2           """ Fonction qui gère les connexions au serveur d'éventuels clients en
                temps réel """
3           while True:
4               client, ip = self.socket.accept()
5               self.clients.append((self.id_client, client, ip))
6               # on lance un fil d'exécution pour chaque nouveau client afin de
                    pouvoir recevoir ses messages
7               # on choisit la bonne fonction de réception en fonction du modèle
                    utilisé
8               if self.modele != 3:
9                   self.threads[str(self.id_client)] = threading.Thread(target=self.
                        reception, args=[client, self.id_client])
10              else:
11                  self.threads[str(self.id_client)] = threading.Thread(target=self.
                        reception_cm, args=[client, self.id_client])
12              self.threads[str(self.id_client)].start()
13
14              print("Le client", self.id_client, "d'adresse", ip, "s'est connecté
                    ")
15              self.id_client += 1
16
17      def transfert(self, expediteur:str, destinataire, message:str, separateur=":"):
18          """ Fonction qui transfert un message entre 2 clients """
19          new_msg = "<Client-" + expediteur + ">" + separateur + message
20          destinataire.send(new_msg.encode("utf-8"))
```

Annexe - serveur.py

```
1       def reception(self, client, id_client:int):
2           """ Fonction qui réceptionne les requêtes des clients et qui les traite
                pour les modèles 1 et 2"""
3           while True:
4               requete_client = client.recv(500)
5               requete_client = requete_client.decode("utf-8")
6               print("<Client-" + str(id_client) + ">:", requete_client)
7               print() # on passe une ligne pour aérer l'affichage
8               if requete_client == "/S␣-close":
9                   print("CLOSE")
10                  self.arret = True
11              else: # on envoie le message au bon destinataire en fonction de l'expé
                    diteur
12                  if id_client == 1:
13                      self.transfert(str(id_client), self.clients[1][1],
                            requete_client, separateur="|")
14                  else:
15                      self.transfert(str(id_client), self.clients[0][1],
                            requete_client, separateur="|")
```

Annexe - serveur.py

```
1       def reception_cm(self, client, id_client:int):
2           """ Fonction qui réceptionne les requêtes des clients et qui les traite
                pour le modèle 3 """
3           while True:
4               requete_client = client.recv(500)
5               requete_client = requete_client.decode("utf-8")
6               decomp_msg = fu.decomposition_chaine(requete_client, separateur="|")
7               print("<Client-" + str(id_client) + ">:", requete_client)
8               print() # on passe une ligne pour aérer l'affichage
9               if requete_client == "/S␣-close":
10                  print("CLOSE")
11                  self.arret = True
12              elif "/S␣-check" in requete_client:
13                  if id_client == 1:
14                      destinataire = 1
15                  else:
16                      destinataire = 0
17                  self.transfert(str(id_client), self.clients[destinataire][1],
                        requete_client, separateur="|")
```

Annexe - serveur.py

```
1                elif "/S␣-init" in requete_client:
2                    if len(decomp_msg) > 2: # si le client malveillant veut envoyer une
                         clé altérée
3                        # on l'envoie au bon destinataire
4                        expediteur = decomp_msg[2]
5                        destinataire = int(decomp_msg[3]) - 1
6                        # on supprime les infos concernant l'expéditeur et le
                             destinataire
7                        decomp_msg.pop()
8                        decomp_msg.pop()
9                        requete_client = fu.compression_arg_str(decomp_msg, separateur=
                             "|")
10                       self.transfert(expediteur, self.clients[destinataire][1],
                             requete_client, separateur="|")
11                   else: # sinon c'est la clé d'origine d'un client normal
12                       # on envoie donc sa clé au client malveillant
13                       self.transfert(str(id_client), self.clients[2][1],
                             requete_client, separateur="|")
14               elif id_client != 3: # si le serveur reçoit un message d'un client
                      normal
15                   # on l'envoit au client malveillant
16                   self.transfert(str(id_client), self.clients[2][1], requete_client,
                          separateur="|")
17               else:
18                   # on l'envoie au bon destinataire
19                   expediteur = decomp_msg[4]
20                   destinataire = int(decomp_msg[5]) - 1
21                   # on supprime les infos concernant l'expéditeur et le destinataire
22                   decomp_msg.pop()
23                   decomp_msg.pop()
24                   requete_client = fu.compression_arg_str(decomp_msg, separateur="|")
25                   self.transfert(expediteur, self.clients[destinataire][1],
```

Annexe - serveur.py

```
1       def run(self):
2           """ Fonction qui démarre et ferme le serveur """
3           # création du socket pour "démarrer" le serveur
4           self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5           self.socket.bind((self.host, self.port))
6           print("Le serveur est démarré...")
7
8           self.socket.listen(1) # on le met en mode récepteur
9
10          connections = threading.Thread(target=self.connections, args=[])
11          connections.start()
12
13          execution = threading.Thread(target=self.close, args=[])
14          execution.start()
15
16          execution.join()
17
18          for client in self.clients:
19              client[1].close()
20          self.socket.close()
```

Annexe - client.py

```python
class Client_RSA:
    """ Classe permettant de simuler un client pouvant se connecter à un serveur informatique simplifié
    à l'aide du module socket, qui base le chiffrement sur l'algorithme du RSA """

    def __init__(self, id: int, liste_nb_premiers, host="localhost", port=6390):
        self.id = id
        self.host = host
        self.port = port
        self.socket = None
        self.cle_pv_RSA = None
        self.cle_pb_RSA = None
        self.cles_pb_RSA_autres = {}
        self.liste_nb_premiers = liste_nb_premiers
        self.init_envoi = False # permet de savoir si self.initialisation_RSA() a déjà été exécutée dans self.envoyer()
        self.init_recu = False  # permet de savoir si self.initialisation_RSA() a déjà été exécutée dans self.reception()
        # messages envoyés de la forme : "Client_X:taille_msg:message_chiffre"
        # On renomme les indices que l'on utilise dans le code pour pouvoir les modifier plus facilement
        # et pour plus de visibilité
        self.i_client = 0
        self.i_taille_msg = 1
        self.i_msg_chiffre = 2
```

Annexe - client.py

```
1       def initialisation_RSA(self, message: str, separateur="|"):
2           """ Fonction qui crée la clé privée et la clé publique du client pour l'
                algo de chiffrement RSA.
3           Elle est exécutée à l'initialisation de la connection avec le serveur et
                partage la clé publique du client
4           avec un autre client connecté au serveur avec lequel il veut communiquer.
                """
5
6           cles = algos.creation_cles(self.liste_nb_premiers)
7           self.cle_pb_RSA = cles[0]
8           self.cle_pv_RSA = cles[1]
9           msg = fu.compression_arg_str([self.cle_pb_RSA[0], self.cle_pb_RSA[1]])
10          msg = message + separateur + msg
11
12          return msg
13
14      def envoyer(self, socket):
15          """ Fonction permettant d'envoyer un message à un client en passant par le
                serveur """
16          try:
17              while True:
18                  msg = input("-->")
19                  if msg == "/S␣-init" and not self.init_envoi:
20                      socket.send(self.initialisation_RSA(msg).encode("utf-8"))
21                      self.init_envoi = True
22                  elif msg == "/S␣-close":
23                      socket.send(msg.encode("utf-8"))
```

Annexe - client.py

```
                else:
                    try:
                        if self.id == 1: # on chiffre le message avec la clé publique du bon client
                            # msg_chiffre = algos.chiffrer_RSA(algos.string_to_int(msg), self.cles_pb_RSA_autres["2"])
                            msg_chiffre = algos.chiffrer_chaine_RSA(msg, self.cles_pb_RSA_autres["2"], taille_facteur=3)
                        else:
                            # msg_chiffre = algos.chiffrer_RSA(algos.string_to_int(msg), self.cles_pb_RSA_autres["1"])
                            msg_chiffre = algos.chiffrer_chaine_RSA(msg, self.cles_pb_RSA_autres["1"], taille_facteur=3)
                        # on transforme le message sous forme de liste en chaine de caractères
                        msg_chiffre = fu.compression_arg_str(msg_chiffre)
                        msg_envoye = str(len(msg)) + "|" + str(msg_chiffre) # on ajoute la longueur du message chiffré au message envoyé
                        msg_envoye = msg_envoye.encode("utf-8")
                        socket.send(msg_envoye)
                    except AssertionError:
                        print("Erreur lors de la tentative de chiffrement du message. Veuillez essayer à nouveau")
            except ConnectionResetError:
                print("Déconnexion du serveur...")
                sys.exit()
```

Annexe - client.py

```
1       def reception(self, socket):
2           """ Fonction qui réceptionne les messages reçus du serveur """
3           try:
4               while True:
5                   requete_serveur = socket.recv(500)
6                   requete_serveur = requete_serveur.decode("utf-8")
7                   # requete_serveur est de la forme : "Client_X:taille_msg:
                        message_chiffre"
8                   decomp_msg = fu.decomposition_chaine(requete_serveur)
9                   id_expediteur = fu.id_client(requete_serveur)
10
11                  if "/S␣-init" in decomp_msg and not self.init_recu: # si on a pas
                        enregistré les clés publiques de l'autre client, on le fait
12                      self.cles_pb_RSA_autres[id_expediteur] = fu.extraction_arg_str(
                            decomp_msg[self.i_msg_chiffre])
13                      self.init_recu = True
14                      if not self.init_envoi: # si on a pas échangé notre clé
                            publique, on le fait
15                          socket.send(self.initialisation_RSA("/S␣-init").encode("utf
                                -8"))
16                          self.init_envoi = True
```

Annexe - client.py

```
1                   else:
2                       taille_msg = int(decomp_msg[self.i_taille_msg])
3                       # on reconvertit le message chiffré (str) en liste d'entiers
                            pour pouvoir ensuite le déchiffrer
4                       msg_chiffre = fu.extraction_arg_str(decomp_msg[self.
                            i_msg_chiffre])
5                       # on vérifie qui est l'expéditeur pour prendre sa clé publique
                            correspondante
6                       if id_expediteur == 1:
7                           # msg_dechiffre = algos.dechiffrer_RSA(int(decomp_msg[2]),
                                self.cle_pv_RSA, self.cle_pb_RSA[0])
8                           msg_dechiffre = algos.dechiffrer_chaine_RSA(msg_chiffre,
                                self.cle_pv_RSA, self.cle_pb_RSA[0], taille_entiers=3)
9                       else:
10                          msg_dechiffre = algos.dechiffrer_chaine_RSA(msg_chiffre,
                                self.cle_pv_RSA, self.cle_pb_RSA[0], taille_entiers=3)
11                      print("\n", decomp_msg[self.i_client] + ":" + msg_dechiffre)
12                  print("\n-->", end="")
13          except ConnectionResetError:
14              print("Connexion interrompue\nSi le terminal est en attente, appuyez
                    sur Entrée")
15              sys.exit()
```

Annexe - client.py

```
1   def connection(self):
2           """ Fonction permettant de connecter le client à un serveur """
3           self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
4           self.socket.connect((self.host, self.port))
5           envoi = threading.Thread(target=self.envoyer, args=[self.socket])
6           recep = threading.Thread(target=self.reception, args=[self.socket])
7           envoi.start()
8           recep.start()
9
10  class Client_AES_RSA(Client_RSA):
11      """ Classe fille de Client_RSA permettant de simuler un client pouvant se
              connecter à un serveur informatique simplifié
12      à l'aide du module socket, qui base le chiffrement sur une combinaison de l'
              algorithme RSA et AES """
13      def __init__(self, id: int, liste_nb_premiers):
14          Client_RSA.__init__(self, id, liste_nb_premiers)
15          # messages envoyés de la forme : "Client_X|taille_msg|cle_AES|vect_AES|
                  message_chiffre"
16          # On renomme les indices que l'on utilise dans le code pour pouvoir les
                  modifier plus facilement
17          # et pour plus de visibilité
18          self.i_client = 0
19          self.i_taille_msg = 1
20          self.i_cle_AES = 2
21          self.i_vect_AES = 3
22          self.i_msg_chiffre = 4
```

Annexe - client.py

```
1       def envoyer(self, socket):
2           """ Fonction permettant d'envoyer un message à un client en passant par le
                serveur """
3           try:
4               while True:
5                   msg = input("-->")
6                   if msg == "/S␣-init" and not self.init_envoi:
7                       # on envoie notre clé publique à l'autre client
8                       socket.send(self.initialisation_RSA(msg, separateur="|").encode
                            ("utf-8"))
9                       self.init_envoi = True
10                  elif msg == "/S␣-close":
11                      socket.send(msg.encode("utf-8"))
12                  elif msg == "/S␣-check":
13                      # on construit le hash de la clé publique de l'expéditeur et on
                            l'envoie à l'autre client
14                      cle_pb = str(self.cle_pb_RSA[0]) + "/" + str(self.cle_pb_RSA
                            [1])
15                      hash_cle = hashlib.sha256(cle_pb.encode())
16                      hash_cle = hash_cle.hexdigest()
17                      msg += "|" + hash_cle
18                      socket.send(msg.encode())
19                  else:
20                      try:
21                          cle_AES = fu.generateur_alea_chaine(32)
22                          vecteur_AES = fu.generateur_alea_chaine(16)
23                          # si on veut envoyer un msg au client 1, on chiffre avec la
                                clé publique du client 2
24                          if self.id == 1:  # on chiffre le message avec la clé
                                publique du bon client
25                              destinataire = "2"
```

Annexe - client.py

```
                        else: # sinon c'est l'inverse
                            destinataire = "1"
                        # On chiffre le message avec la cle et le vecteur AES
                        msg_chiffre = algos.chiffrer_AES(msg, fu.encode_ascii(
                            cle_AES), fu.encode_ascii(vecteur_AES))
                        msg_chiffre = msg_chiffre.decode("latin-1")
                        # On chiffre la cle et le vecteur pour AES avec le RSA
                        cle_AES = algos.chiffrer_chaine_RSA(cle_AES, self.
                            cles_pb_RSA_autres[destinataire], taille_facteur=4)
                        vecteur_AES = algos.chiffrer_chaine_RSA(vecteur_AES, self.
                            cles_pb_RSA_autres[destinataire], taille_facteur=4)
                        # on transforme le message, la clé et le vecteur sous forme
                             de liste en chaine de caractères
                        msg_chiffre = fu.compression_arg_str([msg_chiffre])
                        cle_AES = fu.compression_arg_str(cle_AES)
                        vecteur_AES = fu.compression_arg_str(vecteur_AES)
                        # On construit le message que l'on va envoyer contenant
                            toutes les infos nécessaires pour le destinataire
                        msg_envoye = str(len(msg)) + "|" + cle_AES + "|" +
                            vecteur_AES + "|" + msg_chiffre
                        msg_envoye = msg_envoye.encode("utf-8")
                        socket.send(msg_envoye)
                    except AssertionError:
                        print("Erreur lors de la tentative de chiffrement du
                            message. Veuillez essayer à nouveau")
            except ConnectionResetError:
                print("Déconnexion du serveur...")
                sys.exit()
```

Annexe - client.py

```
1       def reception(self, socket):
2           """ Fonction qui réceptionne les messages reçus du serveur """
3           try:
4               while True:
5                   requete_serveur = socket.recv(500)
6                   requete_serveur = requete_serveur.decode("utf-8")
7                   decomp_msg = fu.decomposition_chaine(requete_serveur, separateur="|
                        ")
8                   id_expediteur = fu.id_client(requete_serveur)
9                   if "/S␣-init" in decomp_msg and not self.init_recu:  # si on a pas
                        enregistré les clés publiques de l'autre client, on le fait
10                      self.cles_pb_RSA_autres[id_expediteur] = fu.extraction_arg_str(
                            decomp_msg[2])
11                      self.init_recu = True
12                      if not self.init_envoi:  # si on a pas échangé notre clé
                            publique, on le fait
13                          socket.send(self.initialisation_RSA("/S␣-init", separateur=
                                "|").encode("utf-8"))
14                          self.init_envoi = True
15                  elif "/S␣-check" in requete_serveur:
16                      print("\nComparaison␣des␣clés␣en␣cours...")
17                      # on construit le hash de la clé de l'autre client à partir de
                            notre base de donnée
18                      cle_pb = str(self.cles_pb_RSA_autres[id_expediteur][0]) + "/" +
                            str(self.cles_pb_RSA_autres[id_expediteur][1])
19                      hash_cle = hashlib.sha256(cle_pb.encode())
20                      hash_cle = hash_cle.hexdigest()
21                      # on récupère le hash envoyé par l'autre client
22                      hash_autre_cle = decomp_msg[-1]
```

Annexe - client.py

```
1                       if hash_autre_cle != hash_cle:
2                           print("La communication n'est plus sécurisée /!\ ")
3                       else:
4                           print("La communication est sécurisée")
5                   else:
6                       # On extrait et déchiffre la clé et le vecteur AES
7                       cle_AES = fu.extraction_arg_str(decomp_msg[self.i_cle_AES])
8                       cle_AES = algos.dechiffrer_chaine_RSA(cle_AES, self.cle_pv_RSA,
                            self.cle_pb_RSA[0], taille_entiers=4)
9                       vecteur_AES = fu.extraction_arg_str(decomp_msg[self.i_vect_AES
                            ])
10                      vecteur_AES = algos.dechiffrer_chaine_RSA(vecteur_AES, self.
                            cle_pv_RSA, self.cle_pb_RSA[0], taille_entiers=4)
11
12                      # on récupère le message chiffré et on le déchiffre
13                      msg_chiffre = decomp_msg[self.i_msg_chiffre]
14                      msg_chiffre = msg_chiffre.encode("latin-1")
15                      msg_dechiffre = algos.dechiffrer_AES(msg_chiffre, fu.
                            encode_ascii(cle_AES), fu.encode_ascii(vecteur_AES))
16                      print("\n", decomp_msg[self.i_client] + ":" + msg_dechiffre)
17                  print("\n-->", end="")
18          except ConnectionResetError:
19              print("Connexion interrompue\nSi le terminal est en attente, appuyez
                    sur Entrée")
20              sys.exit()
```

Annexe - client.py

```python
class ClientMalveillant(Client_AES_RSA):
    """ Classe fille de la classe Client_AES_RSA qui initialise le client malveillant voulant lire des conversations
    sécurisées entre les autres clients connectés au même serveur, dans le cadre du modèle n 3 avec la fonction
     de hachage et le canal secret"""
    def __init__(self, id: int, liste_nb_premiers):
        Client_AES_RSA.__init__(self, id, liste_nb_premiers)
        # Compteur indiquant le nombre d'échanges de clés que le Client Malveillant a effectué avec les autres clients
        self.nb_cles_echangees = 0

    def initialisation_RSA(self, message: str, separateur="|", expediteur="", destinataire=""):
        """ Fonction qui crée la clé privée et la clé publique du client pour l'algo de chiffrement RSA.
        Elle est exécutée à l'initialisation de la connection avec le serveur et partage la clé publique du client
        avec un autre client connecté au serveur avec lequel il veut communiquer.
        """
        if self.cle_pv_RSA == None: # si on a pas encore créé nos clés, on le fait
            cles = algos.creation_cles(self.liste_nb_premiers)
            self.cle_pb_RSA = cles[0]
            self.cle_pv_RSA = cles[1]
        msg = fu.compression_arg_str([self.cle_pb_RSA[0], self.cle_pb_RSA[1]])
        print("clé envoyée : ", msg)
        msg = fu.compression_arg_str([msg, expediteur, destinataire], separateur="|")
        msg = message + separateur + msg
        return msg
```

Annexe - client.py

```
1       def reception(self, socket):
2           """ Fonction qui réceptionne les messages reçus du serveur """
3           try:
4               while True:
5                   requete_serveur = socket.recv(500)
6                   requete_serveur = requete_serveur.decode("utf-8")
7                   decomp_msg = fu.decomposition_chaine(requete_serveur, separateur="|
                        ")
8                   id_expediteur = fu.id_client(requete_serveur)
9
10                  if "/S -init" in decomp_msg:  # si on reçoit une clé publique, on l
                        'enregistre
11                      self.cles_pb_RSA_autres[id_expediteur] = fu.extraction_arg_str(
                            decomp_msg[2])
12                      print("clé reçue :", fu.extraction_arg_str(decomp_msg[2]))
13                      # on envoie notre propre clé publique au bon destinataire
14                      if id_expediteur == "1":
15                          socket.send(self.initialisation_RSA("/S -init", separateur=
                                "|", expediteur=id_expediteur, destinataire="2").
                                encode("utf-8"))
16                      else:
17                          socket.send(self.initialisation_RSA("/S -init", separateur=
                                "|", expediteur=id_expediteur, destinataire="1").
                                encode("utf-8"))
```

Annexe - client.py

```
                else:
                    # On extrait et déchiffre la clé et le vecteur AES
                    cle_AES = fu.extraction_arg_str(decomp_msg[self.i_cle_AES])
                    cle_AES = algos.dechiffrer_chaine_RSA(cle_AES, self.cle_pv_RSA,
                          self.cle_pb_RSA[0], taille_entiers=4)
                    vecteur_AES = fu.extraction_arg_str(decomp_msg[self.i_vect_AES
                         ])
                    vecteur_AES = algos.dechiffrer_chaine_RSA(vecteur_AES, self.
                         cle_pv_RSA, self.cle_pb_RSA[0], taille_entiers=4)

                    # on récupère le message chiffré et on le déchiffre
                    msg_chiffre = decomp_msg[self.i_msg_chiffre]
                    msg_chiffre = msg_chiffre.encode("latin-1")
                    msg_dechiffre = algos.dechiffrer_AES(msg_chiffre, fu.
                         encode_ascii(cle_AES), fu.encode_ascii(vecteur_AES))
                    print("\n", decomp_msg[self.i_client] + ":" + msg_dechiffre)

                    # on renvoie le message au destinataire chiffré avec la clé
                         publique du destinataire
                    # si on veut envoyer un msg au client 1, on chiffre avec la clé
                          publique du client 2
                    if id_expediteur == "1":  # on chiffre le message avec la clé
                         publique du bon client
                        destinataire = "2"
                    else:  # sinon c'est l'inverse
                        destinataire = "1"
```

Annexe - client.py

```
                    # On chiffre le message avec la cle et le vecteur AES
                    msg_chiffre = algos.chiffrer_AES(msg_dechiffre, fu.encode_ascii
                        (cle_AES), fu.encode_ascii(vecteur_AES))
                    msg_chiffre = msg_chiffre.decode("latin-1")
                    # On chiffre la cle et le vecteur pour AES avec le RSA
                    cle_AES = algos.chiffrer_chaine_RSA(cle_AES, self.
                        cles_pb_RSA_autres[destinataire], taille_facteur=4)
                    vecteur_AES = algos.chiffrer_chaine_RSA(vecteur_AES, self.
                        cles_pb_RSA_autres[destinataire], taille_facteur=4)
                    # on transforme le message, la clé et le vecteur sous forme de
                        liste en chaine de caractères
                    msg_chiffre = fu.compression_arg_str([msg_chiffre])
                    cle_AES = fu.compression_arg_str(cle_AES)
                    vecteur_AES = fu.compression_arg_str(vecteur_AES)
                    # On construit le message que l'on va envoyer contenant toutes
                        les infos nécessaires pour le destinataire
                    msg_envoye = str(len(msg_dechiffre)) + "|" + cle_AES + "|" +
                        vecteur_AES + "|" + msg_chiffre + "|" + id_expediteur + "|
                        " + destinataire
                    msg_envoye = msg_envoye.encode("utf-8")
                    socket.send(msg_envoye)
            except ConnectionResetError:
                print("Connexion interrompue\nSi le terminal est en attente, appuyez
                    sur Entrée")
                sys.exit()

        def connection(self):
            """ Fonction permettant de connecter le client à un serveur """
            self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            self.socket.connect((self.host, self.port))
            recep = threading.Thread(target=self.reception, args=[self.socket])
            recep.start()
```

Annexe - fonctions_utiles.py

```python
def copie_dict(dico:dict):
    """ Fonction qui renvoie une copie indépendante du dictionnaire 'dico' """
    dico_copie = {}
    for cle, valeur in dico.items():
        dico_copie[cle] = valeur

    return dico_copie

def pgcd(a: int, b: int):
    """ Fonction qui prend en paramètre 2 entiers naturel a et b et renvoie leur
        PGCD """

    dividende = max(a, b)
    diviseur = min(a, b)
    temp = 0
    while (dividende % diviseur) != 0 :
        temp = dividende % diviseur
        dividende = diviseur
        diviseur = temp

    return diviseur
```

Annexe - fonctions_utiles.py

```python
def compression_arg_str(liste_args: list, separateur="/"):
    """ Fonction qui prend en paramètre une liste d'objets pouvant être convertis
        en chaine de caractères, un séparateur '/' et renvoie une chaine de caract
        ères contenant n paramètres et qui est de la forme 'arg[1]/arg[2]/.../arg[
        n]' """
    arguments = ""
    for i in range(len(liste_args)):
        if i != len(liste_args) - 1:
            arguments += str(liste_args[i]) + separateur
        else:
            arguments += str(liste_args[i])
    return arguments

def extraction_arg_str(chaine_args: str, separateur="/"):
    """ Fonction qui prend en paramètre une chaine de caractères contenant n
        arguments et de la forme 'arg[1]/arg[2]/.../arg[n]', un séparateur '/' et
        renvoie une liste contenant les n arguments de type int"""
    chiffres = "0123456789"
    liste_args = []
    arg_courant = ""
    for i in range(len(chaine_args)):
        if chaine_args[i] != separateur and chaine_args[i] in chiffres:
            arg_courant += chaine_args[i]
        else:
            liste_args.append(int(arg_courant))
            arg_courant = ""
    if arg_courant != "":
        liste_args.append(int(arg_courant))
    return liste_args
```

Annexe - fonctions_utiles.py

```python
def decomposition_chaine(chaine: str, separateur="|"):
    """ Fonction qui prend en paramètre une chaîne de caractères que l'on peut
        diviser en sous-chaînes séparées par un séparateur ':' et renvoie une
        liste contenant ces sous-chaînes """
    liste_sous_chaines = []
    arg_courant = ""
    for i in range(len(chaine)):
        if chaine[i] != separateur:
            arg_courant += chaine[i]
        else:
            liste_sous_chaines.append(arg_courant)
            arg_courant = ""
    if arg_courant != "":
        liste_sous_chaines.append(arg_courant)
    return liste_sous_chaines

def id_client(chaine: str):
    """ Fonction qui prend une chaine de caractères de type '<Client- * > *** ' et
        renvoie l'entier * en chaine de caractères """
    chiffres = "0123456789"
    id_passe = False
    id = ""
    for char in chaine:
        if char in chiffres:
            id_passe = True
            id += char
        elif id_passe:
            break
    return id
```

Annexe - fonctions_utiles.py

```python
def importer_fichier(nom_fichier:str, type="int"):
    """ Importe un fichier texte et ajoute chaque ligne du fichier dans une liste
        Python que la fonction renvoie """
    liste_nb_premiers = []
    with open(nom_fichier, "r") as fichier:
        contenu = fichier.readlines()
    for ligne in contenu:
        ligne_actuelle = ligne.rstrip("\n")
        if type == "int":
            liste_nb_premiers.append(int(ligne_actuelle))
        else:
            liste_nb_premiers.append(ligne_actuelle)
    return liste_nb_premiers

def generateur_alea_chaine(taille_chaine:int):
    """ Fonction qui génère aléatoirement une chaine de caractères de taille '
        taille_chaine', composée des lettres de l'alphabet en minuscule, et la
        renvoie """
    liste_int_a_char = "abcdefghijklmnopqrstuvwxyz"
    nb_char = len(liste_int_a_char)
    chaine_alea = ""
    for _ in range(taille_chaine):
        i = rd.randint(0, nb_char - 1)
        chaine_alea += liste_int_a_char[i]
    return chaine_alea
```

Annexe - fonctions_utiles.py

```python
def concat_chaine(chaine:str, n:int):
    """ Fonction qui renvoie la concaténation de 'chaine' (n-1) fois """

    chaine_finale = ""
    for _ in range(n):
        chaine_finale += chaine

    return chaine_finale

def encode_ascii(chaine:str):
    """ Fonction qui transforme une chaine str en bstr (ie en code ASCII)\n
    NB : elle fait la même chose que la fonction native bytes(chaine, encoding="
        ascii") """

    return bytes(chaine, encoding="utf-8")
```

Annexe - fonctions_utiles.py

```python
def matrice_gf(chaine: str, taille=16):
    """ Renvoie la matrice GF sous forme de matrice de type list[list] """
    matrice_gf = decomposition_chaine(chaine, separateur="␣")
    matrice_gf_bis = []
    matrice_gf_finale = []
    ligne_gf = []

    for i in range(len(matrice_gf)):
        if "\n" in matrice_gf[i]:
            matrice_gf_bis.append(matrice_gf[i][:(-1)])
        elif matrice_gf[i] != "":
            matrice_gf_bis.append(matrice_gf[i])

    for i in range(taille):
        for j in range(taille):
            ligne_gf.append(matrice_gf_bis[i*taille + j])
            if j == taille - 1:
                matrice_gf_finale.append(ligne_gf)
                ligne_gf = []

    return matrice_gf_finale
```

Annexe - fonctions_utiles.py

```python
def conversion_matrice(matrice: list[list], fonction, base=10):
    """ Fonction qui applique la fonction de conversion 'fonction' sur chaque case
        de la matrice carrée """
    res = matrice.copy()
    for i in range(len(matrice)):
        for j in range(len(matrice)):
            if base != 10:
                res[i][j] = fonction(res[i][j], base)
            else:
                res[i][j] = fonction(res[i][j])

    return res
```

Annexe - fonctions_utiles.py

```python
def produit_matriciel(m1: list[list[int]], m2: list[list[int]], n1=(-1, -1), n2
    =(-1, -1)):
    """ Fonction qui renvoie le produit matriciel des matrices m1 et m2 """
    assert(n1[1] == n2[0])
    if n1 == (-1,-1):
        n = len(m1)
        p = n
        q = n
    else:
        n, p = n1[0], n1[1]
        q = n2[1]
    res = [[0 for _ in range(q)] for _ in range(n)]
    for i in range(n):
        for j in range(q):
            somme = 0
            for k in range(p):
                somme += m1[i][k] * m2[k][j]
            res[i][j] = somme
    return res

def somme_matricielle(m1: list[list[int]], m2: list[list[int]], n: int, p: int):
    """ Fonction qui renvoie la somme des matrices m1 et m2 """
    res = [[0 for _ in range(p)] for _ in range(n)]
    for i in range(n):
        for j in range(p):
            res[i][j] = m1[i][j] + m2[i][j]
    return res
```

Annexe - fonctions_utiles.py

```python
def bin_vers_dec(x:str):
    """ Fonction qui convertit un élément de type int binaire en élément de type
        int décimal"""
    x_str = str(x)
    x_liste = [int(c) for c in x_str]
    x_liste.reverse()
    x_int = 0
    k = 0
    for c in x_liste:
        x_int += 2**k * c
        k += 1

    return x_int

def remplir_zeros(x, n: int):
    """ Rajoute des zéros jusqu'à ce que 'x' soit un nombre binaire codé sur n bits
        """
    x_str = str(x)
    while len(x_str) < n:
        x_str = "0" + x_str

    return x_str
```

Annexe - fonctions_utiles.py

```python
def affiche_matrice(m: list[list]):
    """ Fonction qui affiche une matrice carrée """
    for i in range(len(m)):
        for j in range(len(m)):
            print(m[i][j], "␣", end="")
        print()

def decale_liste(liste: list, d: int):
    """ Fonction qui décale les éléments de la liste de 'd' vers la droite
    et renvoie le résultat"""
    liste_bis = liste.copy()
    liste_finale = []
    d_bis = d

    if d == 0:
        return liste

    while d_bis > 0:
        liste_finale = []
        liste_finale.append(liste_bis.pop())
        for el in liste_bis:
            liste_finale.append(el)
        liste_bis = liste_finale.copy()
        d_bis -= 1

    return liste_finale
```